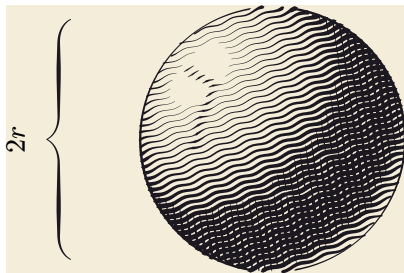


solids3d

3D solids, knots and engraved figures for Typst — one function: `fig()`

Reference manual Version 0.4.0 · Typst ≥ 0.15 · LGPL-3.0-or-later

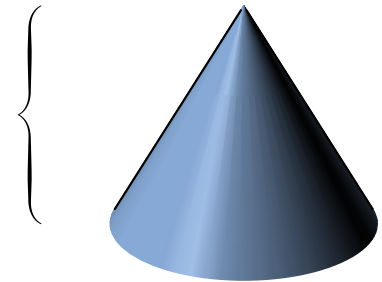
Author FERGOUS Abdelhak <@fergousA>



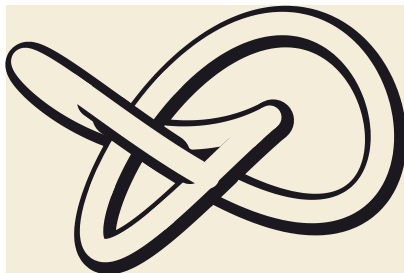
`fig(sphere(1), style: "banknote")`



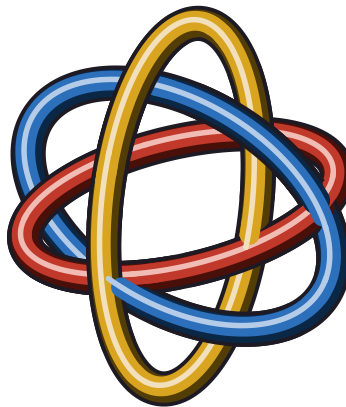
`fig(torus(1, 0.38), style: "engraving")`



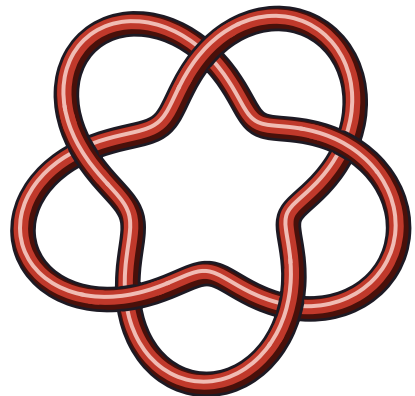
`fig(cone(1, 1.8), brace3(...))`



`fig(trefoil(), style: "engraving")`



`fig(..borromean())`



`fig(torus-knot(2, 5), view: "top")`

Contents

Getting started	4
What solids3d does	4
Installation	4
Your first figure	4
The simple API at a glance	5
Choosing the view	5
Choosing the style	5
Dimensions with braces	5
Knots and links	6
An engraved plate	6
Reference — the simple layer	7
fig	7
view-projection	8
paint	8
cube / brick	9
Reference — solids and surfaces	10
Standard solids	10
Surfaces and meshes	13
Unit solids and boxes	16
Reference — points, paths and transforms	17
Vectors	17
Building paths	17
Operations on paths	19
Transforms	20
Reference — the advanced layer: picture, draw, camera, light	21
picture	21
Drawing commands	22
Light, materials and pens	26
Reference — styles: engraving, vintage, pencil	28
Engraving	28
Vintage and pencil	29
Other tools	32
Reference — knots, braces and plates	34
Knots and links	34
Plates	36
Reference — school geometry	38
Solids for lessons	38
Annotations	39
Appendices	43
Performance and memory	43
Troubleshooting	43
Coming from Asymptote	43
What's new in 0.3	43
Licences and credits	44

Index of functions

brace3.....	35
callout3.....	40
circle3 / arc3 / arc3-angles.....	18
cone.....	11
construction3 / height3 / diagonal3 / projection3 41	
cuboid / regular-pyramid / regular-prism.....	38
cuboid-vertices / cuboid-edges.....	42
cylinder.....	10
dimension3.....	39
dot.....	24
draw.....	22
edge-code3 / equal-edges3.....	40
engraved.....	29
engraving.....	28
extrude / cone-over.....	13
fill3.....	24
graph3.....	18
guide3.....	18
heightfield.....	14
knot3.....	34
label.....	24
light.....	26
limits.....	25
line3.....	17
material.....	26
mesh-edges.....	15
obj.....	32
orthographic / perspective.....	22
pen & friends.....	27
pencil-pen.....	31
picture.....	21
plane.....	19
plate.....	36
point3 / vertex3.....	39
regular-tetrahedron / regular-octahedron / frustum3 / right-prism3.....	38
revolution.....	12
right-angle3 / angle-mark3 / angle-code3.....	41
shift / scale3 / rotate3 / reflect3.....	20
silhouette.....	23
solid-plot.....	32
sphere.....	10
spline3.....	17
surface / patch.....	15
surface-grid.....	13
surface-mesh.....	14
torus.....	11
trefoil / figure-eight / torus-knot / hopf-link / borromean.....	34
tube3.....	12
unit solids.....	16
vintage-hatch.....	29
vintage-picture / pencil-picture.....	31
vintage-shaded / vintage-material.....	30
vintage-stipple.....	30
xaxis3 / yaxis3 / zaxis3 / axes3.....	25

Getting started

What solids3d does

`solids3d` draws three-dimensional figures directly in a Typst document: solids of revolution (sphere, cone, cylinder, torus), cubes and blocks, parametric surfaces and meshes, 3D curves, **knots and links**, calligraphic **braces**, and school-geometry annotations (dimensions, right angles, equal-edge marks). Every figure is an ordinary vector box: it scales, it can be put in a table, a `figure`, a grid, and the PDF stays vector.

The package is built around **one** function, `fig`. You give it what you want to see; it chooses the camera, the light, the shading, the outlines and the page-ready size. For anything finer, the full `picture` / `draw` machinery inspired by Asymptote's `three` and `solids` modules is available (chapters “advanced layer”).

Since 0.3 the pens, outlines, knots and braces are drawn by `nibart` (MIT).

Installation

```
#import "@preview/solids3d:0.4.0": *
```

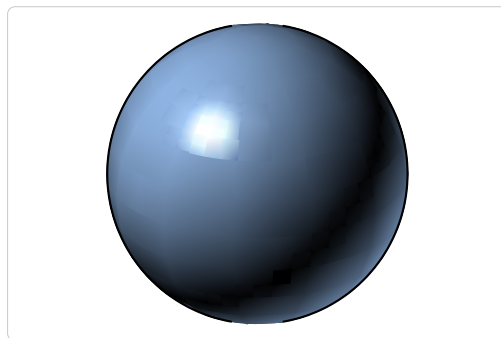
Typst Universe edition: nothing to install, Typst downloads the package (and `@preview/nibart:0.3.0`) on first use. A local edition (zip with an installer) installs the same package under `@local`.

The star import brings in a lot of short names (`draw`, `light`, `plane`, `sphere`...). If a name clashes with yours, import only what you need: `#import "...": fig, sphere, torus, brace3`.

Your first figure

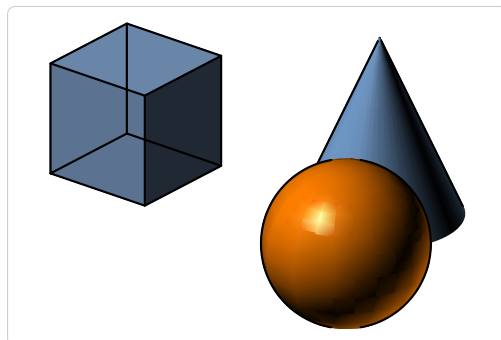
One line: a solid inside `fig`. That is all.

```
// a lit, outlined sphere
fig(sphere(1), size: 4cm)
```



Several solids: they share the same camera and light. Colours come from `paint(shape, colour)`.

```
fig(
  cube(1, c: (0, -1.8, 0.5)), // c: = centre
  paint(cone(0.7, 1.6, c: (-0.6, 0.3, 0)),
  rgb("#7aa6d8")),
  paint(sphere(0.7, c: (1.2, 1.4, 0.7)), orange),
  size: 5.5cm)
```



The simple API at a glance

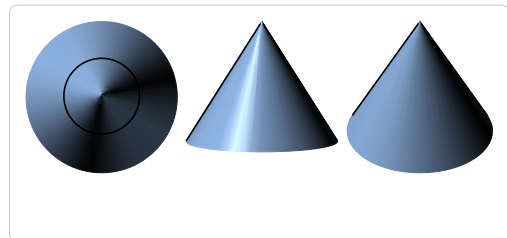
Five ideas are enough for most figures.

You write	You get
<code>fig(...items, size, view, style)</code>	a finished figure: camera, light, shading and outlines chosen for you
<code>sphere(r)</code> <code>cone(r, h)</code> <code>cylinder(r, h)</code> <code>torus(R, r)</code> <code>cube(a)</code> <code>brick(l, w, h)</code>	the solids (option <code>c:</code> = centre)
<code>paint(shape, colour)</code>	a coloured solid
view: <code>"iso"</code> <code>"front"</code> <code>"top"</code> <code>"side"</code> <code>"low"</code> <code>"high"</code> (az, el)	the camera
style: <code>"shaded"</code> <code>"engraving"</code> <code>"banknote"</code> <code>"etching"</code> <code>"vintage"</code> <code>"pencil"</code>	the look: Phong shading, copper-plate engraving, wavy banknote lines, blue-ink etching, old-plate hatching, pencil
<code>trefoil()</code> <code>torus-knot(p, q)</code> <code>hopf-link()</code> <code>borromean()</code>	knots and links: just give them to <code>fig</code>
<code>brace3(a, b, label)</code>	a calligraphic brace placed outside the figure
<code>plate(figure, title: ..., caption: ...)</code>	an engraved frame around a figure

Choosing the view

view accepts a name or a pair (azimuth, elevation) in degrees. `persp: true` switches to a perspective camera.

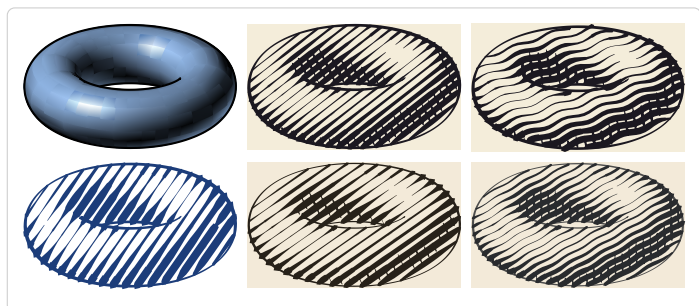
```
let s = cone(1, 1.6)
grid(columns: 3, gutter: 4pt,
  fig(s, size: 2.7cm, view: "top"),
  fig(s, size: 2.7cm, view: "front"),
  fig(s, size: 2.7cm, view: (60, 35), persp: true))
```



Choosing the style

The same scene in every style. `engraving` and `banknote` replace the shading by a screen of lines whose thickness follows the tone; `etching` is the same in blue ink on white paper.

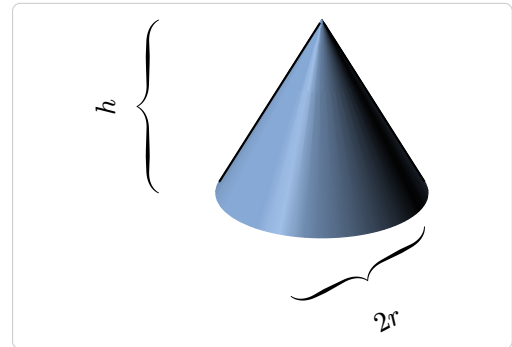
```
let scene(st) = fig(torus(1, 0.38),
  size: 2.9cm, style: st)
grid(columns: 3, gutter: 4pt,
  scene("shaded"),
  scene("engraving"),
  scene("banknote"),
  scene("etching"), scene("vintage"),
  scene("pencil"))
```



Dimensions with braces

`brace3(a, b, label)` joins two 3D points. The brace is placed on the outside of the figure by itself.

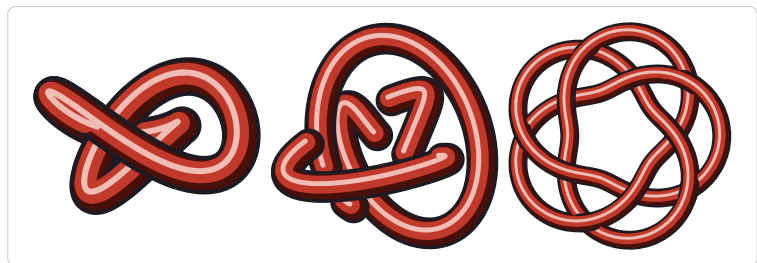
```
fig(cone(1, 1.8), size: 4.4cm,
    brace3((0, 0, 0), (0, 0, 1.8), $h$),
    brace3((-1, 0, 0), (1, 0, 0), $2r$))
```



Knots and links

Give curves to `fig`: they become tubes, interrupted or woven at the crossings. `tube` is the width, `knot-style` is "weave" (default) or "gap".

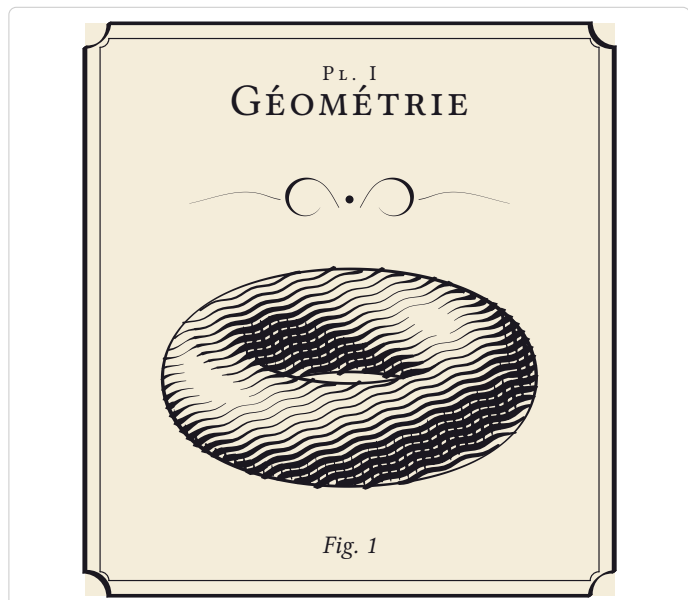
```
grid(columns: 3, gutter: 4pt,
    fig(trefoil(), size: 3cm, tube:
    10pt),
    fig(figure-eight(), size: 3cm,
    tube: 8pt,
    knot-style: "gap", view: (60,
    50)),
    fig(torus-knot(3, 5, R: 2, r:
    0.8), size: 3cm,
    tube: 6pt, view: "top"))
```



An engraved plate

`plate` wraps any content (a figure, a grid of figures) in a frame with notched corners and a calligraphic flourish.

```
plate(title: "Géométrie", number: "I",
    width: 7cm,
    caption: [Fig. 1],
    fig(torus(1, 0.38), size: 5cm,
    style: "banknote"))
```



Reference — the simple layer

This chapter documents the functions of the **simple layer**: `fig` and its helpers. Everything else (the following chapters) is the advanced layer that `fig` is built on; you never need it for ordinary figures.

fig

```
fig(..items, size: 6cm, view: "iso", style: "shaded", light: auto, rim: auto, persp:
false, tube: 7pt, knot-style: "weave", engraving: auto, ..picture-options)
```

Turns what you give it into a finished figure. Each **item** is converted by its nature: a solid of revolution or a surface is lit and outlined; a 3D path is drawn as a line; a knot curve (`trefoil()`, a function `t => (x, y, z)`) becomes a tube — all the curves given in one call form **one** link, so that their crossings are detected; `paint(shape, colour)` gives a colour; a command (`brace3(...)`, `draw(...)`, `knot3(...)`, `dimension3(...)` ...) is passed through untouched. The camera comes from `view`, the light follows the camera (it comes from the upper left).

PARAMETERS

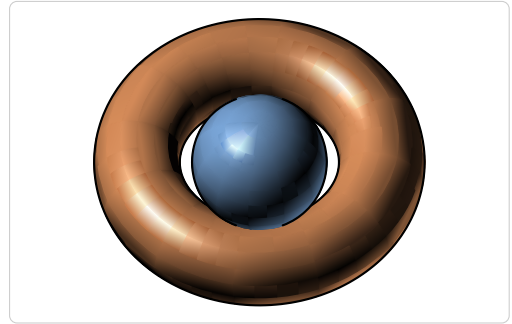
name	type	default	description
<code>..items</code>	shape, surface, path, curve or command	required	What to draw (any number, any mix).
<code>size</code>	length	6cm	Width of the figure (the height follows the content).
<code>view</code>	string or array or projection	"iso"	"iso", "front", "back", "side", "left", "top", "bottom", "low", "high"; or (azimuth, elevation) in degrees; or a projection built with orthographic / perspective.
<code>style</code>	string	"shaded"	"shaded" (Phong shading), "engraving" (copper-plate), "banknote" (wavy engraving), "etching" (blue-ink engraving), "vintage", "pencil".
<code>light</code>	light	auto	A light made with <code>light(...)</code> ; <code>auto</code> = from the upper left of the camera.
<code>rim</code>	pen or none	auto	Pen of the outlines of the solids (<code>auto</code> = 0.8 pt black); <code>none</code> removes them.
<code>persp</code>	bool	false	Perspective camera instead of orthographic.
<code>tube</code>	length	7pt	Diameter of the knot tubes.
<code>knot-style</code>	string	"weave"	"weave" (continuous tubes) or "gap" (the under-strand is interrupted).
<code>engraving</code>	dictionary	auto	An <code>engraving(...)</code> to tune the lines of the engraving, banknote and etching styles.
<code>..picture-options</code>	any	required	Any other option is passed to <code>picture: shading, backend, zsort, refine, engine, fill, margin, limits, unitsize...</code>

RETURNS a box (content).

NOTE Items are drawn in the order given: put the solids that must stay in front last. Knots form one layer, drawn after the solids.

EXAMPLE

```
fig(  
  paint(torus(1, 0.35), rgb("#d98e5b")),  
  paint(sphere(0.55), rgb("#7aa6d8")),  
  size: 4.4cm, view: "high")
```



view-projection

```
view-projection(v, persp: false)
```

The projection used by `fig` for a named view or a pair of angles. Use it to share one camera between `fig` and `picture`.

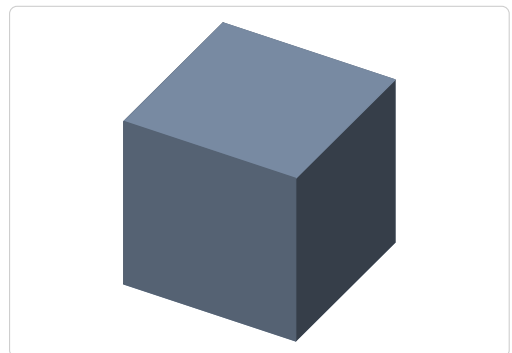
PARAMETERS

name	type	default	description
<code>v</code>	string, array or projection	required	Name of a view, (azimuth, elevation) in degrees, or a projection (returned unchanged).
<code>persp</code>	bool	false	Perspective instead of orthographic.

RETURNS a projection.

EXAMPLE

```
let P = view-projection((30, 35))  
picture(width: 3.6cm, projection: P,  
  light: light((-1, 0.8, 1.2)),  
  draw(surface(unitcube), rgb("#9db4d4")))
```



paint

```
paint(shape, color)
```

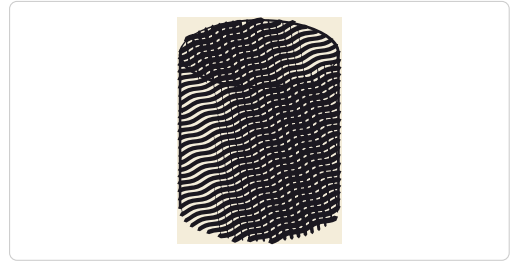
Gives a colour to a solid or a surface inside `fig`. In the engraving styles the colour sets the tone (dark colours give more ink).

PARAMETERS

name	type	default	description
<code>shape</code>	shape or surface	required	The solid.
<code>color</code>	color	required	Diffuse colour.

EXAMPLE

```
fig(Paint(cylinder(0.7, 1.5), rgb("#e8a33d")),
      size: 3cm, style: "banknote")
```



cube / brick

```
cube(a, c: (0, 0, 0))    brick(l, w, h, c: (0, 0, 0))
```

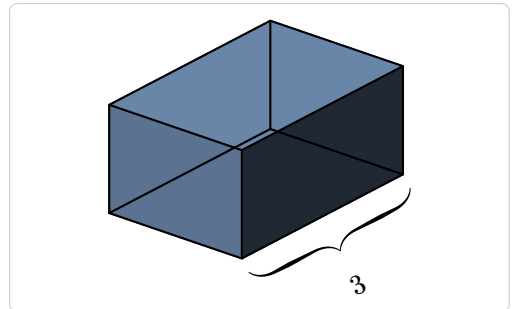
A cube of side `a`, or a rectangular block of length `l` (x), width `w` (y) and height `h` (z), centred on `c`. They are flat-shaded surfaces; `fig` also draws their edges (hidden ones dashed).

PARAMETERS

name	type	default	description
<code>a</code>	number	required	Side of the cube.
<code>l, w, h</code>	number	required	Length, width and height of the block.
<code>c</code>	point	<code>(0, 0, 0)</code>	Centre.

EXAMPLE

```
fig(brick(3, 2, 1.4), size: 4cm,
     brace3((-1.5, 1, -0.7), (1.5, 1, -0.7), $3$))
```



Reference — solids and surfaces

Constructors of solids and surfaces. Each one returns a value that you can give to `fig` (simple layer) or to `draw(surface(...))` (advanced layer). Solids of revolution also support skeletons and silhouettes (see `draw` and `silhouette`).

Standard solids

sphere

```
sphere(r)      sphere(c, r)      sphere(c, r, n)      sphere(..., n: 12, c: auto)
```

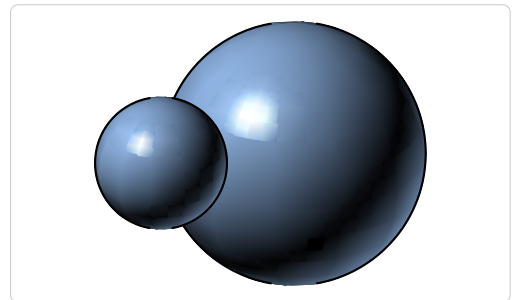
Sphere of radius `r`, as a solid of revolution about the z axis. Three call forms: radius only (centred at the origin), centre and radius, or centre, radius and number of Bézier segments in the half circle.

PARAMETERS

name	type	default	description
<code>r</code>	number	required	Radius.
<code>c</code>	point	<code>(0, 0, 0)</code>	Centre (positional first argument, or the named option <code>c</code> :).
<code>n</code>	integer	<code>12</code>	Bézier segments in the half circle (smoothness).

EXAMPLE

```
fig(sphere(1), sphere(0.5, c: (1.6, 0, 0.5)), size: 4.4cm)
```



cylinder

```
cylinder(r, h)      cylinder(c, r, h, axis)      cylinder(..., axis: Z, c: auto)
```

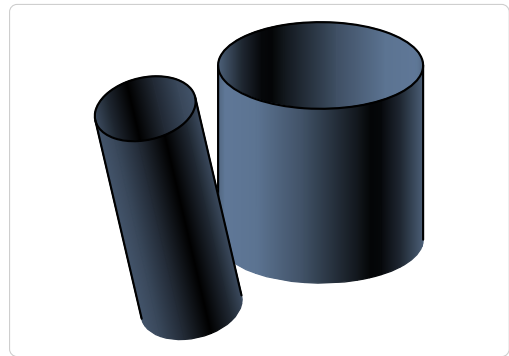
Cylinder of radius `r` and height `h` whose base circle is centred on `c`; `axis` is the direction of the axis.

PARAMETERS

name	type	default	description
<code>r</code>	number	required	Radius.
<code>h</code>	number	required	Height.
<code>c</code>	point	<code>(0, 0, 0)</code>	Centre of the base.
<code>axis</code>	point	<code>Z</code>	Direction of the axis.

EXAMPLE

```
fig(cylinder(0.8, 1.5), cylinder(0.4, 2, c: (1.6, 0, 0), axis: (0.3, 0, 1)), size: 4.4cm)
```



cone

```
cone(r, h)      cone(c, r, h, axis, n)      cone(..., axis: Z, n: 12, c: auto)
```

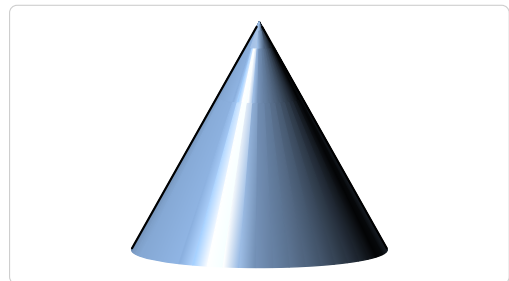
Cone with base radius `r`, height `h`, base centre `c`.

PARAMETERS

name	type	default	description
<code>r</code>	number	required	Radius of the base.
<code>h</code>	number	required	Height.
<code>c</code>	point	(0, 0, 0)	Centre of the base.
<code>axis</code>	point	Z	Direction of the axis.
<code>n</code>	integer	12	Number of slices (smoothness).

EXAMPLE

```
fig(cone(1, 1.8), size: 3.4cm, view: "low")
```



torus

```
torus(R, a, c: (0, 0, 0), n: 16)
```

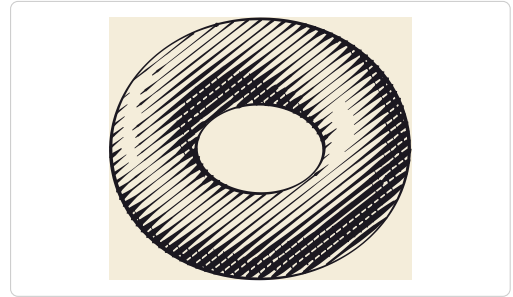
Torus of major radius `R` (distance from the centre to the tube axis) and minor radius `a` (tube radius), about the `z` axis. Its silhouette is exact, with the hidden parts of the hole removed.

PARAMETERS

name	type	default	description
<code>R</code>	number	required	Major radius.
<code>a</code>	number	required	Minor radius.
<code>c</code>	point	(0, 0, 0)	Centre.
<code>n</code>	integer	16	Segments of the tube circle.

EXAMPLE

```
fig(torus(1, 0.4), size: 4cm, view: "high", style:
"engraving")
```



revolution

```
revolution(g, axis)      revolution(c, g, axis)      revolution(..., c: auto, axis: Z,
angle1: 0, angle2: 360)
```

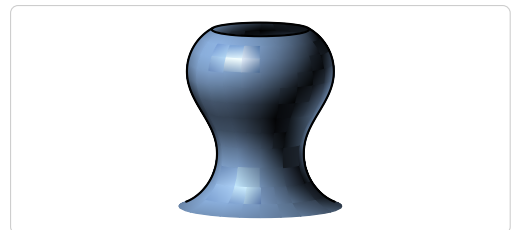
The general solid of revolution: the 3D path `g` turned about `axis` (a direction through `c`). `sphere`, `cylinder`, `cone` and `torus` are special cases. `angle1`, `angle2` limit the sweep, in degrees.

PARAMETERS

name	type	default	description
<code>g</code>	<code>path</code>	required	Generatrix (a 3D path, e.g. made with <code>line3</code> or <code>spline3</code>).
<code>axis</code>	<code>point</code>	<code>Z</code>	Direction of the axis of revolution.
<code>c</code>	<code>point</code>	<code>(0, 0, 0)</code>	A point of the axis.
<code>angle1, angle2</code>	<code>number</code>	<code>0, 360</code>	Start and end angles of the sweep, in degrees.

EXAMPLE

```
let vase = spline3((1, 0, 0), (0.55, 0, 0.8), (0.9,
0, 1.6), (0.6, 0, 2.2))
fig(revolution(vase, Z), size: 2.6cm, view:
"front")
```



tube3

```
tube3(g, width: 0.1, n: 12, samples: auto, caps: false, color: none) (alias tube)
```

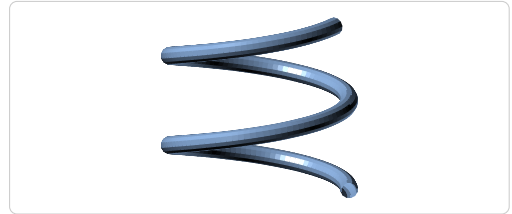
A polygonal tube around a 3D path, built from a transported frame — helices, wires, rods.

PARAMETERS

name	type	default	description
<code>g</code>	<code>path</code>	required	Spine of the tube.
<code>width</code>	<code>number</code>	<code>0.1</code>	Diameter of the tube.
<code>n</code>	<code>integer</code>	<code>12</code>	Number of sides.
<code>samples</code>	<code>integer</code>	<code>auto</code>	Subdivisions per Bézier segment.
<code>caps</code>	<code>bool</code>	<code>false</code>	Close the ends.
<code>color</code>	<code>color or function</code>	<code>none</code>	Colour, or <code>(i, th) => colour</code> .

EXAMPLE

```
let helix = spline3(..range(25).map(i =>
  (calc.cos(i * 0.5), calc.sin(i * 0.5), i * 0.08)))
fig(tube3(helix, width: 0.18), size: 2.6cm, view:
  "front")
```



extrude / cone-over

```
extrude(p, v: Z, n: 8)      cone-over(base, vertex, n: 4)
```

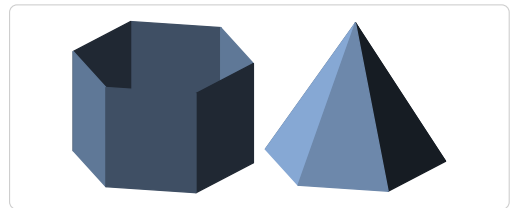
`extrude` sweeps a planar path along the vector `v` (a prism, a cylinder with any section). `cone-over` joins a base path to a vertex (pyramid, cone with any base).

PARAMETERS

name	type	default	description
<code>p / base</code>	<code>path</code>	required	The planar path / the base.
<code>v</code>	<code>point</code>	<code>Z</code>	Extrusion vector.
<code>vertex</code>	<code>point</code>	required	Apex of the cone.
<code>n</code>	<code>integer</code>	<code>8 / 4</code>	Subdivisions.

EXAMPLE

```
let hexa = line3(..range(6).map(i => (calc.cos(i *
  60deg), calc.sin(i * 60deg), 0)), cyclic: true)
grid(columns: 2, gutter: 4pt,
  fig(extrude(hexa, v: (0, 0, 1.2)), size: 2.4cm),
  fig(cone-over(hexa, (0, 0, 1.6)), size: 2.4cm))
```



Surfaces and meshes

surface-grid

```
surface-grid(f, u0, u1, v0, v1, nu: 16, nv: 12, cyclic-u: false, cyclic-v: false,
  normal: auto, color: none)      (aliases parametric-surface, parametric)
```

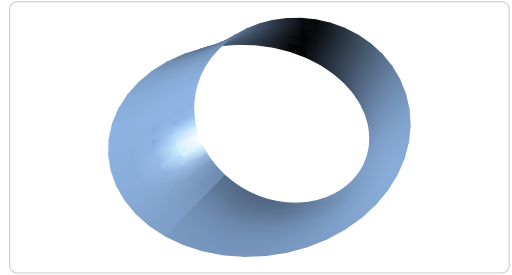
Parametric surface: `f(u, v)` returns a point. The domain is cut into `nu × nv` patches.

PARAMETERS

name	type	default	description
<code>f</code>	<code>function</code>	required	<code>(u, v) => (x, y, z)</code> .
<code>u0, u1, v0, v1</code>	<code>number</code>	required	Parameter ranges.
<code>nu, nv</code>	<code>integer</code>	<code>16, 12</code>	Number of patches along <code>u</code> and <code>v</code> .
<code>cyclic-u, cyclic-v</code>	<code>bool</code>	<code>false</code>	Close the surface in that direction (tori, cylinders).
<code>normal</code>	<code>auto, point or function</code>	<code>auto</code>	Normal: automatic, constant, or <code>(u, v) => point</code> .
<code>color</code>	<code>color or function</code>	<code>none</code>	Colour, or <code>(u, v) => colour</code> evaluated at the centre of each patch.

EXAMPLE

```
let mobius = surface-grid(
  (u, v) => ((1 + v * calc.cos(u / 2)) *
    calc.cos(u),
    (1 + v * calc.cos(u / 2)) *
    calc.sin(u), v * calc.sin(u / 2)),
  0, 2 * calc.pi, -0.4, 0.4, nu: 48, nv: 4, cyclic-
    u: false)
fig(mobius, size: 4cm, view: "high")
```



heightfield

```
heightfield(f, x0, x1, y0, y1, nx: 16, ny: 16, normal: auto, color: none)
```

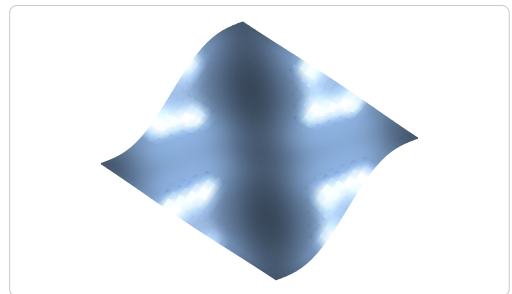
Surface $z = f(x, y)$ over a rectangle.

PARAMETERS

name	type	default	description
<code>f</code>	function	required	<code>(x, y) => z</code> .
<code>x0, x1, y0, y1</code>	number	required	The rectangle.
<code>nx, ny</code>	integer	16, 16	Patches along x and y.
<code>normal, color</code>	...	auto, none	As in <code>surface-grid</code> .

EXAMPLE

```
fig(heightfield((x, y) => 0.35 * calc.sin(2 * x) *
  calc.cos(2 * y), -1.6, 1.6, -1.6, 1.6, nx: 28, ny:
  28),
  size: 4.2cm, view: "high")
```



surface-mesh

```
surface-mesh(vertices, faces, indices: "zero", normal: auto, smooth: true, color: none)
(alias mesh-surface)
```

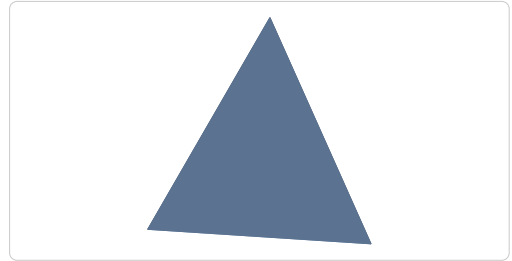
Surface from an indexed mesh (polyhedra, `.obj`-like data). Faces are lists of vertex indices, triangles or polygons.

PARAMETERS

name	type	default	description
<code>vertices</code>	array	required	List of points.
<code>faces</code>	array	required	List of index lists.
<code>indices</code>	string	"zero"	"zero" or "one" : numbering of the indices.
<code>smooth</code>	bool	true	Average the normals at the vertices; false = flat faces (polyhedra).
<code>normal, color</code>	...	auto, none	Constant or function normal; colour or <code>(face, centre) => colour</code> .

EXAMPLE

```
let v = ((0, 0, 0), (1.4, 0, 0), (0.7, 1.2, 0),
(0.7, 0.4, 1.3))
fig(surface-mesh(v, ((0, 2, 1), (0, 1, 3), (1, 2,
3), (2, 0, 3)), smooth: false), size: 3cm)
```



mesh-edges

```
mesh-edges(vertices, faces, indices: "zero", boundary: false)
```

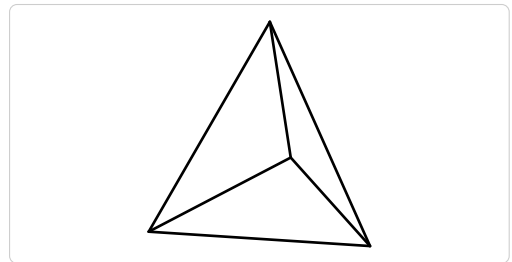
The unique edges of a mesh, as 3D paths, to draw a wireframe: `draw(mesh-edges(v, f), pen)`.

PARAMETERS

name	type	default	description
vertices, faces, indices	...	required	As in <code>surface-mesh</code> .
boundary	bool	false	Keep only the edges of a single face (the border).

EXAMPLE

```
let v = ((0, 0, 0), (1.4, 0, 0), (0.7, 1.2, 0),
(0.7, 0.4, 1.3))
let f = ((0, 2, 1), (0, 1, 3), (1, 2, 3), (2, 0,
3))
fig(draw(mesh-edges(v, f), linewidth(1pt)), size:
3cm)
```



surface / patch

```
surface(..args, n: 12, color: none, normal: auto, planar: false) patch(boundary,
normal: auto, center: auto, color: none, holes: ())
```

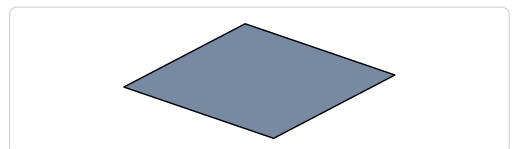
`surface(x)` converts a solid of revolution, a closed planar path or a list of patches into a surface that `draw` can shade. `patch` builds one four-sided or polygonal patch from its boundary path.

PARAMETERS

name	type	default	description
args	shape, path or patches	required	What to convert (several surfaces are merged).
n	integer	12	Slices for solids of revolution.
color	color or function	none	Colour of the patches.
normal	auto or point	auto	Normal.
planar	bool	false	The path is planar (fill it as one patch).
boundary	path	required	Boundary path of a patch.
holes	array	()	Holes of a patch (paths).

EXAMPLE

```
picture(width: 3.6cm, projection: view-
projection("iso"),
draw(surface(unitsquare3), rgb("#9db4d4")),
draw(unitsquare3, black))
```



Unit solids and boxes

unit solids

```
unitsphere  unithemisphere  unitcylinder  unitcone  unitsolidcone  unitcube  unitdisk
unitplane  unitfrustum(ta, tb)  unitbox  box3(v1, v2)  unitcircle3  unitsquare3
```

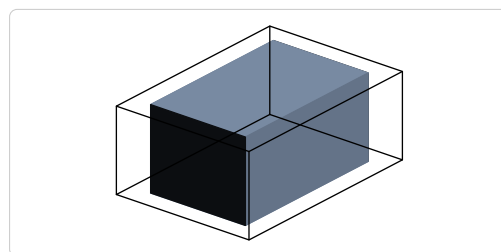
Ready-made unit shapes (from Asymptote's `solids`): the unit sphere, hemisphere, cylinder and cone (radius 1, height 1), the solid cone, the unit cube, disk, plane, the frustum of ratio `ta : tb`. `box3(v1, v2)` is the wireframe box between two opposite corners (hidden edges dashed); `unitbox` is `box3(0, (1, 1, 1))`. Scale and move them with transforms (`shift`, `scale3`, `rotate3`, `apply`).

PARAMETERS

name	type	default	description
<code>ta, tb</code>	number	required	Radii of the frustum at the top and at the bottom (unit height).
<code>v1, v2</code>	point	required	Opposite corners of the box.

EXAMPLE

```
picture(width: 3.8cm, projection: view-
projection("iso"),
  draw(surface(apply(scale3(1.6, y: 1, z: 0.8),
unitcube)), rgb("#9db4d4")),
  draw(box3((-0.2, -0.2, 0), (1.8, 1.2, 0.8)),
black))
```



Reference — points, paths and transforms

A **point** is an array `(x, y, z)`. A **path** (`path3`) is a chain of Bézier segments in space, with Asymptote's Hobby-spline semantics. A **transform** is a 4×4 matrix. Constants: `0 = (0, 0, 0)`, `X`, `Y`, `Z` (unit vectors).

Vectors

function	result
<code>vadd(a, b)</code> <code>vsub(a, b)</code> <code>vmul(k, v)</code> <code>vneg(v)</code>	sum, difference, scalar product, opposite
<code>vdot(a, b)</code> <code>vcross(a, b)</code>	dot product, cross product
<code>vabs(v)</code> <code>vunit(v)</code> <code>vsum(..vs)</code>	norm, unit vector, sum of several
<code>interp(a, b, t)</code>	point $a + t(b - a)$
<code>vmin</code> <code>vmax</code> <code>vproject(u, v)</code> <code>vperp(v)</code> <code>vangle(a, b)</code>	component-wise min / max, projection of <code>u</code> on <code>v</code> , a perpendicular vector, angle in degrees
<code>vdir(theta, phi)</code> <code>expi(theta, phi)</code> <code>colatitude</code> <code>latitude</code> <code>longitude</code>	spherical coordinates (angles in degrees)

Building paths

line3

```
line3(..points, cyclic: false)
```

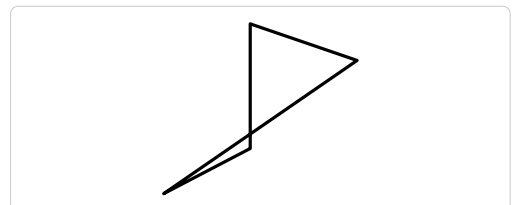
Polyline through the points (straight segments).

PARAMETERS

name	type	default	description
<code>..points</code>	<code>point</code>	required	Vertices.
<code>cyclic</code>	<code>bool</code>	<code>false</code>	Close the path.

EXAMPLE

```
fig(draw(line3(0, X, X + Y, (0, 1, 1), (0, 0, 1),  
cyclic: true), linewidth(1.2pt)), size: 2.6cm)
```



spline3

```
spline3(..points, cyclic: false)
```

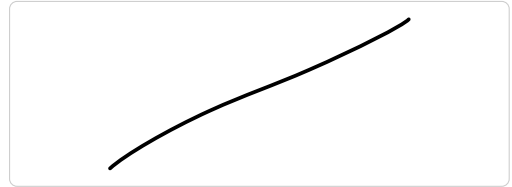
Smooth Hobby spline through the points: the natural way to describe a curve by a few points.

PARAMETERS

name	type	default	description
<code>..points</code>	<code>point</code>	required	Points to interpolate.
<code>cyclic</code>	<code>bool</code>	<code>false</code>	Closed curve.

EXAMPLE

```
fig(draw(spline3((0, 0, 0), (1, 0.6, 0.5), (2, 0, 1), (3, 0.8, 1.4)), linewidth(1.4pt)), size: 4cm, view: "front")
```



guide3

```
guide3(..items, join: "--")
```

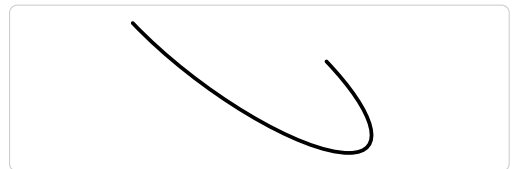
General guide with Asymptote's join operators between nodes: `\"--\"` straight, `\"..\"` Hobby curve. Items are points; `join` sets the operator used between them.

PARAMETERS

name	type	default	description
<code>..items</code>	point	required	Nodes.
<code>join</code>	string	<code>"--"</code>	<code>"--"</code> or <code>".."</code> .

EXAMPLE

```
fig(draw(guide3(0, (1, 1, 0), (2, 0, 1), join: ".."), linewidth(1.4pt)), size: 3.4cm)
```



circle3 / arc3 / arc3-angles

```
circle3(c, r, normal: Z)    arc3(c, v1, v2, normal: 0, direction: CCW)    arc3-angles(c, r, θ1, φ1, θ2, φ2, normal: 0, direction: auto)
```

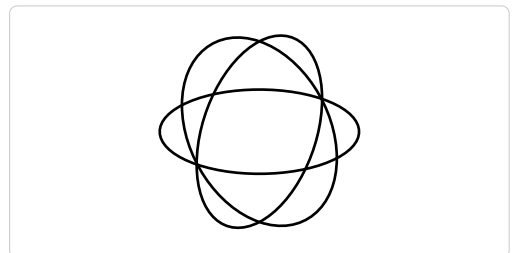
Circle of centre `c`, radius `r` in the plane normal to `normal`. `arc3` is the arc from `v1` to `v2` seen from `c`. `arc3-angles` gives the ends in spherical angles (colatitude θ , longitude ϕ , in degrees).

PARAMETERS

name	type	default	description
<code>c</code>	point	required	Centre.
<code>r</code>	number	required	Radius.
<code>normal</code>	point	<code>Z / 0</code>	Normal to the plane (<code>0</code> = deduced from the ends).
<code>v1, v2</code>	point	required	Ends of the arc.
<code>direction</code>	<code>CCW</code> or <code>CW</code>	<code>CCW</code>	Orientation of the arc.

EXAMPLE

```
fig(draw(circle3(0, 1, normal: X), linewidth(1pt)), draw(circle3(0, 1, normal: Y), linewidth(1pt)), draw(circle3(0, 1), linewidth(1pt)), size: 3cm)
```



graph3

```
graph3(f, a, b, n: 100, join: "--")
```

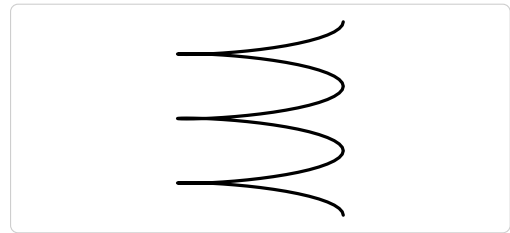
Path of the curve $t \mapsto f(t)$ sampled at n points between a and b .

PARAMETERS

name	type	default	description
<code>f</code>	function	required	$t \Rightarrow (x, y, z)$.
<code>a, b</code>	number	required	Interval.
<code>n</code>	integer	100	Samples.
<code>join</code>	string	"--"	".." gives a smooth spline.

EXAMPLE

```
fig(draw(graph3(t => (calc.cos(t), calc.sin(t), t /
8), 0, 6 * calc.pi, n: 120), linewidth(1.2pt)),
size: 2.6cm, view: "front")
```



plane

```
plane(u, v, o: 0)
```

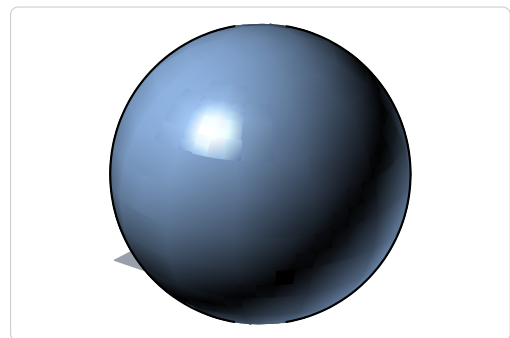
A parallelogram (closed path) spanned by u and v from the origin o : a handy way to draw a plane section.

PARAMETERS

name	type	default	description
<code>u, v</code>	point	required	Edge vectors.
<code>o</code>	point	0	Corner.

EXAMPLE

```
fig(draw(surface(plane(2 * X, 2 * Y)),
rgb("#cfd8e8")), sphere(0.8, c: (1, 1, 0.8)), size:
4cm)
```



Operations on paths

function	result
<code>point(p, t)</code> <code>dir(p, t)</code>	position / unit tangent at time t (integer = node)
<code>size3(p)</code> <code>length3(p)</code> <code>cyclic(p)</code>	number of nodes / segments, is it closed
<code>arclength(p)</code> <code>arctime(p, s)</code> <code>reltime(p, r)</code> <code>midpoint(p)</code>	length; time at arc length s ; time at fraction r of the length; midpoint
<code>subpath(p, a, b)</code> <code>reverse(p)</code>	portion between times a and b ; reversed

function	result
<code>join(p, q)</code> <code>join-all(..ps)</code> <code>close3(p)</code>	concatenation; closing
<code>min3(p)</code> <code>max3(p)</code>	corners of the bounding box
<code>flatten3(p, n: 8)</code> <code>nodes3(p)</code>	polyline approximation; list of the nodes

Transforms

`shift` / `scale3` / `rotate3` / `reflect3`

```
shift(v)  scale3(x, y: auto, z: auto)  xscale3(x)  yscale3(y)  zscale3(z)  rotate3(a, u,
v: none)  reflect3(u, v, w)  align3(u)  compose(..Ts)  apply(T, object)
```

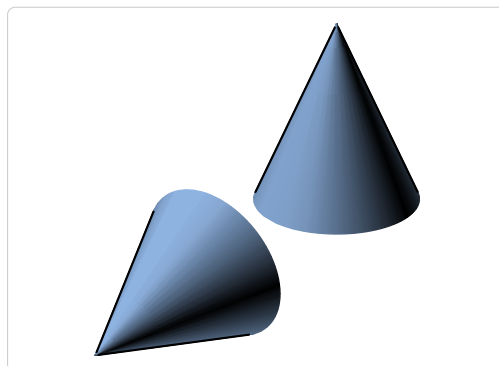
Transforms are 4×4 matrices; `compose(T1, T2)` applies `T2` first. `apply(T, obj)` transforms a point, a path, a surface, a solid of revolution. `rotate3(a, u)` rotates by `a` degrees about the axis through the origin with direction `u` (or through `v` and direction `u` if `v` is given).

PARAMETERS

name	type	default	description
<code>v</code>	point	required	Translation vector (<code>shift(x, y, z)</code> also works).
<code>x, y, z</code>	number	required	Scale factors (<code>y</code> and <code>z</code> default to <code>x</code>).
<code>a</code>	number	required	Angle in degrees.
<code>u</code>	point	required	Axis direction.
<code>T</code>	transform	required	A matrix.

EXAMPLE

```
let c = cone(0.6, 1.4)
fig(c, apply(compose(shift(1.4, 0, 0), rotate3(100,
Y)), c), size: 4.4cm)
```



Reference — the advanced layer: picture, draw, camera, light

`fig` is a thin layer over `picture`. Use `picture` directly when you need full control: several pens, axes, labels, dots, your own camera and light, `zsort`, `perspective`... Everything is a **command**: `draw`, `dot`, `label`, `xaxis3`, `brace3`, `knot3`... return lists of commands that `picture` executes in order.

picture

picture

```
picture(..commands, size: auto, width: auto, height: auto, unitsize: 1cm, projection:
currentprojection, light: currentlight, limits: none, margin: 0pt, inset: 0pt, fill:
none, stroke: none, radius: 0pt, dotfactor: 6, arrowsize: auto, clip: false, baseline:
0pt, zsort: "command", shading: "smooth", refine: true, backend: "svg", style: "none",
engine: "nibart", engraving: auto)
```

Renders commands into a box. Sizing: `width` and/or `height` (or `size`, the larger side) fit the picture; without any, `unitsize` is the length of one unit of the projected plane.

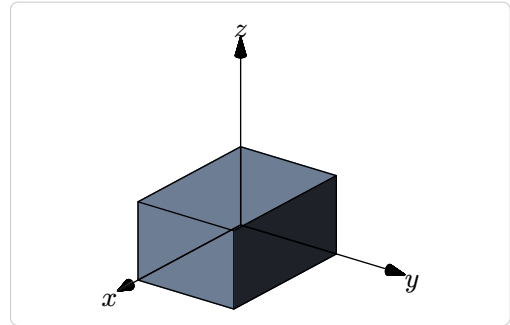
PARAMETERS

name	type	default	description
<code>..commands</code>	<code>commands</code>	required	Lists returned by <code>draw</code> , <code>dot</code> , <code>label</code> , axes, <code>brace3</code> , <code>knot3</code> ...
<code>size, width, height</code>	<code>length</code>	<code>auto</code>	Final size of the figure.
<code>unitsize</code>	<code>length</code>	<code>1cm</code>	Length of one unit when no size is given.
<code>projection</code>	<code>projection</code>	<code>perspective(5, 4, 2)</code>	Camera (see <code>orthographic</code> , <code>perspective</code>).
<code>light</code>	<code>light</code>	<code>Headlamp</code>	Light of the scene.
<code>limits</code>	<code>limits</code>	<code>none</code>	Fixes the 3D box used to frame the camera: <code>limits((x0, y0, z0), (x1, y1, z1))</code> . The same box gives identical framing to several figures.
<code>margin, inset</code>	<code>length</code>	<code>0pt</code>	Space added around the drawing / inside the frame.
<code>fill, stroke, radius</code>	<code>...</code>	<code>none, none, 0pt</code>	Background, border and corner radius of the frame.
<code>dotfactor</code>	<code>number</code>	<code>6</code>	Dot diameter = <code>dotfactor</code> × pen width.
<code>arrowsize</code>	<code>length</code>	<code>auto</code>	Size of the arrowheads.
<code>clip</code>	<code>bool</code>	<code>false</code>	Clip the drawing to the frame.
<code>baseline</code>	<code>length</code>	<code>0pt</code>	Baseline shift of the box.
<code>zsort</code>	<code>string</code>	<code>"command"</code>	<code>"command"</code> : surfaces painted in command order (fast, predictable); <code>"global"</code> : all patches merged and sorted by depth (use when solids interpenetrate).
<code>shading</code>	<code>string</code>	<code>"smooth"</code>	<code>"smooth"</code> , <code>"flat"</code> or <code>"none"</code> .
<code>refine</code>	<code>bool</code>	<code>true</code>	Subdivide patches that cross the silhouette for cleaner edges.
<code>backend</code>	<code>string</code>	<code>"svg"</code>	<code>"svg"</code> (one image) or <code>"typst"</code> (native shapes: selectable, lighter in memory for big drawings).
<code>style</code>	<code>string</code>	<code>"none"</code>	<code>"none"</code> , <code>"vintage"</code> , <code>"pencil"</code> or <code>"engraving"</code> (see chapter styles).
<code>engine</code>	<code>string</code>	<code>"nibart"</code>	Pen engine: only <code>"nibart"</code> (pure Typst + nibart); the <code>"wasm"</code> engine was removed in 0.4.0.
<code>engraving</code>	<code>dictionary</code>	<code>auto</code>	Engraving settings for <code>style: "engraving"</code> .

RETURNS a box.

EXAMPLE

```
picture(width: 4.2cm, projection: orthographic(4,
3, 2.2),
  draw(surface(cuboid(length: 2, width: 1.4,
height: 1)), rgb("#9db4d4"), light((-1, 0.8,
1.2))),
  draw(box3(0, (2, 1.4, 1)), black),
  axes3(min: 0, max: 2.4))
```



orthographic / perspective

```
orthographic(x, y, z)    perspective(x, y, z)    (... , camera: none, up: Z, target: 0,
zoom: 1)
```

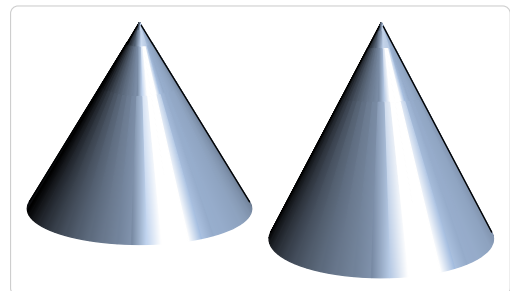
Cameras. The three numbers (or a point) give the position of the eye; **target** is the point looked at, **up** the vertical direction of the page. Orthographic keeps parallels parallel (technical drawing); perspective converges. Ready-made: `LeftView`, `RightView`, `FrontView`, `BackView`, `TopView`, `BottomView`.

PARAMETERS

name	type	default	description
x, y, z	number	required	Position of the camera (only its direction matters in orthographic).
camera	point	none	Same, as a single point.
up	point	Z	Which 3D direction is vertical on the page.
target	point	0	Point looked at.
zoom	number	1	Zoom factor.

EXAMPLE

```
let s = cone(0.8, 1.4)
grid(columns: 2, gutter: 6pt,
  picture(width: 3cm, projection: orthographic(4,
2, 1.5), draw(surface(s), rgb("#9db4d4")),
draw(silhouette(s), black)),
  picture(width: 3cm, projection: perspective(3,
1.5, 1.2), draw(surface(s), rgb("#9db4d4")),
draw(silhouette(s), black)))
```



Drawing commands

draw

```
draw(object, ..pens, pen: auto, arrow: none, label: none, align: auto, light: auto,
meshpen: none, m: auto, n: 12, frontpen: auto, backpen: auto, longitudinalpen: auto,
longitudinalbackpen: auto, ninterpolate: auto, seam: auto, hatch: auto, stipple: auto)
```

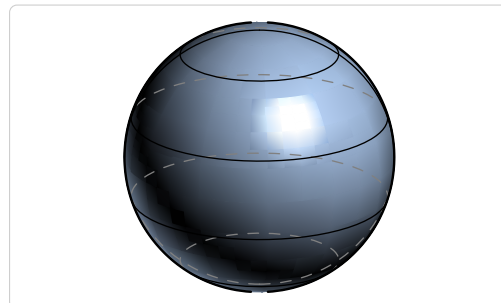
Draws a path (as a line), a surface (shaded), a solid of revolution (its skeleton: meridians and parallels, front pen / back pen), or a deferred object like `silhouette(...)`. Positional extras are classified by type: a colour or pen = the pen, a `Label` = the label, an arrow, a light. Asymptote-style calls work: `draw(Label("x"), path, red, Arrow3)`.

PARAMETERS

name	type	default	description
<code>object</code>	path, surface, shape...	required	What to draw.
<code>pen</code>	pen or colour or material	<code>auto</code>	Line pen, or surface colour/material. For a surface an array gives one material per patch.
<code>arrow</code>	arrow	<code>none</code>	<code>Arrow3</code> , <code>BeginArrow</code> , <code>Arrows</code> ... for a path.
<code>label</code>	content or Label	<code>none</code>	Label attached to the path (<code>Label(body, position, 0.5, align: N)</code>).
<code>align</code>	direction	<code>auto</code>	Label alignment (<code>N</code> , <code>S</code> , <code>E</code> , <code>W</code> , <code>NE</code> ...).
<code>light</code>	light	<code>auto</code>	Light for this surface (e.g. <code>nolight</code>).
<code>meshpen</code>	pen	<code>none</code>	Draws the patch boundaries (wireframe).
<code>m, n</code>	integer	<code>auto</code> , <code>12</code>	Number of meridians / slices of a skeleton.
<code>frontpen, backpen</code>	pen	<code>auto</code>	Visible / hidden parts of a skeleton.
<code>longitudinalpen, longitudinalbackpen</code>	pen	<code>auto</code>	Pens of the meridians.
<code>ninterpolate</code>	integer	<code>auto</code>	Projection subdivisions of a path (curves in perspective).
<code>hatch, stipple</code>	hatch / stipple	<code>auto</code>	Vintage hatching or stippling of a surface (see <code>vintage-hatch</code> , <code>vintage-stipple</code>).

EXAMPLE

```
let s = sphere(1)
picture(width: 3.6cm, projection: orthographic(4,
2, 1.6),
  draw(surface(s), rgb("#a9c4e8")),
  draw(s, 6, 8, frontpen: pen(linewidth(0.5pt),
black), backpen: pen(dashed, gray(0.5))),
  draw(silhouette(s), linewidth(0.9pt)))
```



silhouette

```
silhouette(shape, m: 64)
```

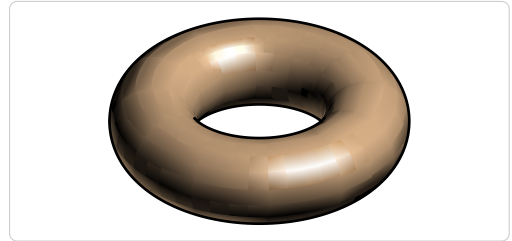
The outline of a solid of revolution as seen by the camera — computed when `picture` knows the projection (hence a deferred object). Exact for spheres, cones, cylinders and tori.

PARAMETERS

name	type	default	description
<code>shape</code>	<code>revolution</code>	required	The solid.
<code>m</code>	integer	<code>64</code>	Samples of the contour.

EXAMPLE

```
let t = torus(1, 0.4)
picture(width: 4cm, projection: orthographic(4, 2, 3),
  draw(surface(t), rgb("#d8b087")),
  draw(silhouette(t), linewidth(1pt)))
```



fill3

```
fill3(path, pen: auto)
```

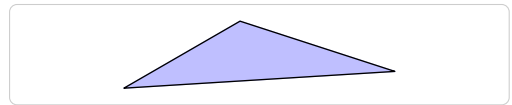
Fills a planar closed path with a flat colour, no lighting.

PARAMETERS

name	type	default	description
path	path	required	Planar closed path.
pen	colour	auto	Fill colour.

EXAMPLE

```
picture(width: 3.6cm, projection: orthographic(4, 3, 2.4),
  fill3(line3(0, X, X + Y, Y, cyclic: true),
  paleblue),
  draw(line3(0, X, X + Y, Y, cyclic: true), black))
```



dot

```
dot(point, pen: auto, label: none, align: auto, size: auto)
```

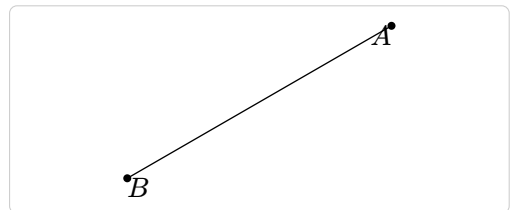
Draws a dot at a point (or at the nodes of a path), with an optional label.

PARAMETERS

name	type	default	description
point	point or path	required	Position(s).
pen	pen	auto	Colour / size.
label	content	none	Label.
align	direction	E	Where the label sits.
size	length	auto	Dot diameter.

EXAMPLE

```
picture(width: 3.6cm, projection: orthographic(4, 3, 2.4),
  draw(line3(0, X, X + Y), black),
  dot(0, label: $A$, align: SW), dot(X, label: $B$,
  align: SE), dot(X + Y, label: $C$, align: NE))
```



label

```
label(body, point, ..args, align: auto, pen: auto)
```

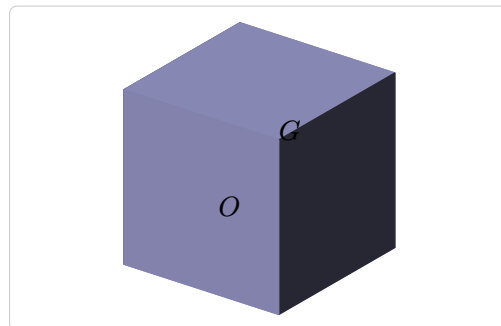
Text (or math) anchored at a 3D point, not rotated.

PARAMETERS

name	type	default	description
<code>body</code>	<code>content</code>	<code>required</code>	The text.
<code>point</code>	<code>point</code>	<code>required</code>	Anchor.
<code>align</code>	<code>direction</code>	<code>Center</code>	Placement around the anchor.
<code>pen</code>	<code>pen</code>	<code>auto</code>	Colour and size.

EXAMPLE

```
picture(width: 3.6cm, projection: orthographic(4,
3, 2.4),
  draw(surface(unitcube), paleblue, light((-1, 0.8,
1.2))), label($O$, 0, align: SW), label($G$, (1, 1,
1), align: NE))
```



`xaxis3` / `yaxis3` / `zaxis3` / `axes3`

```
xaxis3(label, pen, arrow, min: auto, max: auto)    axes3(xlabel: $$, ylabel: $$,
zlabel: $$, pen: auto, arrow: Arrow3, min: auto, max: auto)
```

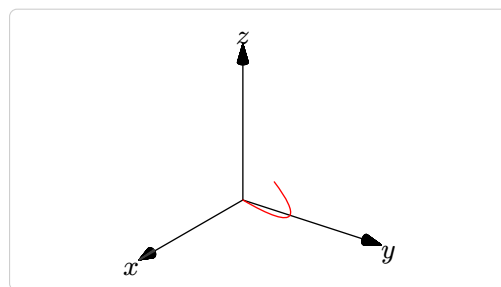
Coordinate axes. `min` / `max` are the coordinates where the axis starts and ends; `auto` takes the scene bounds.

PARAMETERS

name	type	default	description
<code>label</code>	<code>content</code>	<code>none</code>	Axis label.
<code>pen</code>	<code>pen</code>	<code>auto</code>	Pen.
<code>arrow</code>	<code>arrow</code>	<code>none</code> (<code>axes3</code> : <code>Arrow3</code>)	Arrowhead.
<code>min, max</code>	<code>number</code>	<code>auto</code>	Start and end coordinate of the axis.
<code>xlabel, ylabel, zlabel</code>	<code>content</code>	<code>\$\$ \$y\$ \$z\$</code>	Labels for <code>axes3</code> .

EXAMPLE

```
picture(width: 3.6cm, projection: orthographic(4,
3, 2.4),
  axes3(min: 0, max: 2.2), draw(spline3(0, (1, 1.5,
0.6), (2, 2, 1.6)), red))
```



`limits`

```
limits(min, max)
```

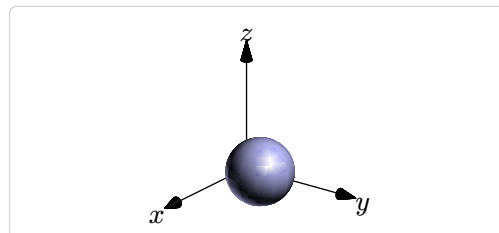
Declares the 3D box of the scene: it gives the extent of the axes drawn with `axes3` (when `min` / `max` are `auto`) and the box used to place the camera. Without it, the bounds of the drawn objects are used. To give several figures the same scale, give them the same `unitsize` instead of a `width`.

PARAMETERS

name	type	default	description
<code>min, max</code>	<code>point</code>	<code>required</code>	Opposite corners of the box.

EXAMPLE

```
picture(unitsize: 0.9cm, projection:
orthographic(4, 3, 2),
limits(0, (2, 2, 2)), axes3(),
draw(surface(sphere(0.5, c: (1, 1, 0.5))),
paleblue))
```



Light, materials and pens

light

```
light(..positions, diffuse: auto, specular: auto, specularfactor: 1, background: none,
viewport: true)
```

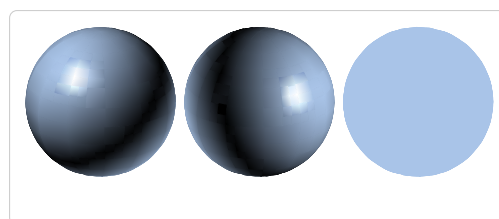
A light source. `light((x, y, z))` gives the direction **the light comes from**; with `viewport: true` (default) the direction is relative to the camera (x to the right, y up, z toward you), with `false` it is fixed in the scene. Positional colours come first: `light(white, gray(0.5), (1, 1, 2))`. `Headlamp` is the default; `nolight` disables shading (flat colours).

PARAMETERS

name	type	default	description
<code>..positions</code>	<code>point</code>	<code>required</code>	Directions (several = several lights).
<code>diffuse</code>	<code>colour</code>	<code>white</code>	Diffuse colour (or an array, one per light).
<code>specular</code>	<code>colour</code>	<code>auto</code>	Colour of highlights.
<code>specularfactor</code>	<code>number</code>	<code>1</code>	Strength of highlights.
<code>background</code>	<code>colour</code>	<code>none</code>	Ambient background colour.
<code>viewport</code>	<code>bool</code>	<code>true</code>	Direction relative to the camera.

EXAMPLE

```
let s = sphere(1)
let pic(L) = picture(width: 2.4cm, projection:
orthographic(4, 2, 1.6), light: L, draw(surface(s),
rgb("#a9c4e8")))
grid(columns: 3, gutter: 4pt, pic(light((-1, 1,
1))), pic(light((1, 0.2, 0.6))), pic(nolight))
```



material

```
material(diffuse: black, emissive: black, specular: mediumgray, opacity: auto,
shininess: 0.7)
```

Surface material: colours, transparency (`opacity`) and glossiness (`shininess`). A plain colour is accepted wherever a material is expected. `withopacity(colour, 0.5)` and `opacity(0.5)` build translucent colours/pens.

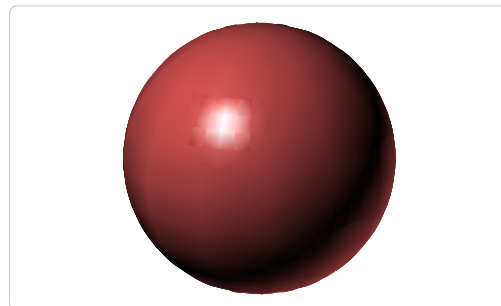
PARAMETERS

name	type	default	description
<code>diffuse</code>	<code>colour</code>	<code>black</code>	Base colour.

name	type	default	description
<code>emissive</code>	<code>colour</code>	<code>black</code>	Self-lit colour added (glow).
<code>specular</code>	<code>colour</code>	<code>mediumgray</code>	Highlight colour.
<code>opacity</code>	<code>number</code>	<code>auto</code>	0 – 1.
<code>shininess</code>	<code>number</code>	<code>0.7</code>	Sharpness of the highlight.

EXAMPLE

```
let s = sphere(1)
picture(width: 3.6cm, projection: orthographic(4,
2, 1.6), light: light((-1, 1, 1.5)),
  draw(surface(s), material(diffuse:
rgb("#d05050"), specular: white, shininess: 0.9)))
```



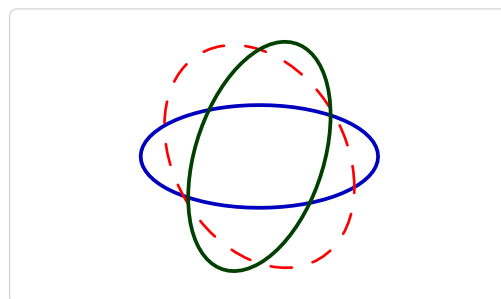
pen & friends

```
pen(..items) linewidth(w) dashed dotted longdashed dashdotted linetype(pattern)
roundcap squarecap roundjoin miterjoin invisible fontsize(s) nolight opacity(x)
withopacity(c, x)
```

Pens are small dictionaries combined with `pen(...): pen(linewidth(1pt), red, dashed)`. Colours are Asymptote's: `red green blue cyan magenta yellow black white orange purple...` and the families `pale...`, `light...`, `medium...`, `heavy...`, `deep...`, `dark...`; `gray(0.4)`; `rgbf(r, g, b)` (0–1). Standard Typst `rgb(...)` also works.

EXAMPLE

```
picture(width: 3.6cm, projection: orthographic(4,
3, 2.4),
  draw(circle3(0, 1), pen(linewidth(1.4pt),
heavyblue)),
  draw(circle3(0, 1, normal: X),
pen(linewidth(1pt), red, dashed)),
  draw(circle3(0, 1, normal: Y),
pen(linewidth(1.4pt), darkgreen, roundcap)))
```



Reference — styles: engraving, vintage, pencil

Three look-and-feel families are available through `picture(style: ...)`; `fig` sets them for you (`style: "engraving", "banknote", "etching", "vintage", "pencil"`). **Engraving** turns tone into the thickness of parallel lines (copper-plate engraving, banknotes); **vintage** engraves surfaces like an old scientific plate (a light sepia line screen); **pencil** does the same in wavy graphite and draws every stroke with a rough graphite pen. In `vintage` and `pencil`, a surface with no material, `hatch:` or `stipple:` is engraved by the lit line screen (`engraving:` tunes it); an explicit `hatch:`, `stipple:` or material gives the projected-patch hatching described below.

Engraving

engraving

```
engraving(spacing: 2.1pt, angle: 38, cross: true, cross-angle: 78, wave: none, gain: 1.0, minimum: 0.13pt, contours: true, nib-width: 1.35, nib-angle: 38, nib-ratio: 0.36, ink: auto, paper: auto, step: 1.6pt)
```

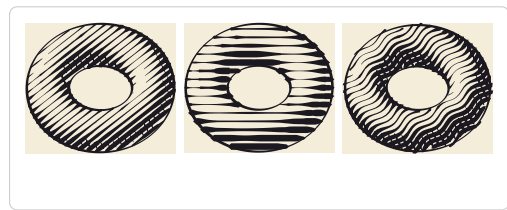
Settings of the engraving screen. Pass the result to `picture(style: "engraving", engraving: engraving(...))` or to `fig(..., engraving: ...)`. The surface is covered by parallel lines whose **thickness** (not spacing) follows the lighting, drawn with a broad nib; a second family crosses them in the deep shadows. The `banknote` style of `fig` is `wave: (...)`; `etching` changes ink and paper.

PARAMETERS

name	type	default	description
<code>spacing</code>	length	<code>2.1pt</code>	Distance between two lines.
<code>angle</code>	number	<code>38</code>	Direction of the first family, in degrees.
<code>cross</code>	bool	<code>true</code>	Add the crossed family in the deepest shadows.
<code>cross-angle</code>	number	<code>78</code>	Offset of the second family from <code>angle</code> .
<code>wave</code>	none or dictionary	<code>none</code>	<code>none</code> = straight lines; <code>(amplitude: 0.5pt, length: 14pt, shift: 0.0)</code> = wavy lines (banknote). A non-zero <code>shift</code> turns the phase from line to line (moiré).
<code>gain</code>	number	<code>1.0</code>	Quantity of ink: <code>> 1</code> darker, <code>< 1</code> lighter.
<code>minimum</code>	length	<code>0.13pt</code>	Thinnest visible line; below it the line disappears (white highlights).
<code>contours</code>	bool	<code>true</code>	Broad-nib outlines of the surfaces.
<code>nib-width, nib-angle, nib-ratio</code>	number	<code>1.35, 38, 0.36</code>	Width factor, angle (degrees) and flatness of the contour nib.
<code>ink, paper</code>	colour	<code>auto</code>	Ink and paper colours (defaults: warm black on cream).
<code>step</code>	length	<code>1.6pt</code>	Sampling step along a line (smaller = smoother, slower).

EXAMPLE

```
let s = torus(1, 0.4)
let e(opts) = fig(s, size: 2.6cm, view: "high",
style: "engraving", engraving: engraving(..opts))
grid(columns: 3, gutter: 4pt,
  e(()),
  e((angle: 0, cross: false, spacing: 3pt)),
  e((wave: (amplitude: 0.7pt, length: 12pt), gain:
1.3)))
```



engraved

```
engraved(shape, m: 48, rim: auto, edges: none, color: none)
```

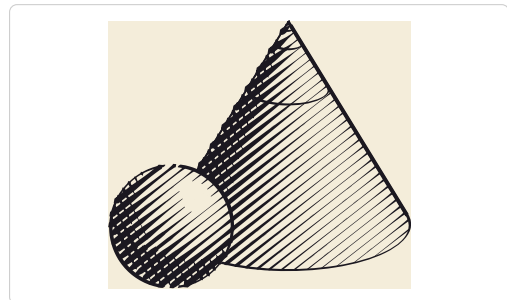
One-call engraved solid for `picture(style: "engraving")`: the hatched surface, its silhouette, and optionally the edges (cone base, box edges). It is what `fig` uses.

PARAMETERS

name	type	default	description
<code>shape</code>	<code>shape</code>	required	Solid of revolution.
<code>m</code>	<code>integer</code>	<code>48</code>	Samples of the silhouette.
<code>rim</code>	<code>pen or none</code>	<code>auto</code>	Pen of the contour (<code>auto</code> = 0.8 pt); <code>none</code> = no contour.
<code>edges</code>	<code>pen or none</code>	<code>none</code>	Pen for the wireframe edges.
<code>color</code>	<code>colour</code>	<code>none</code>	Surface colour (sets the tone).

EXAMPLE

```
picture(width: 4cm, projection: orthographic(4, 3,
2), style: "engraving",
  engraved(cone(1, 1.8), edges: linewidth(0.6)),
  engraved(sphere(0.5, c: (1.6, 0, 0.5))))
```



Vintage and pencil

vintage-hatch

```
vintage-hatch(spacing: 0.24, angle: 35.0, cross: false, broken: true, lit: true,
segment: 0.22, gap: 0.08, light: auto, count: none, length: none, crosshatch: none,
seed: 11, pen: ...)
```

Hatching specification given to `draw(surface, hatch: ...)`. Dashes are broken and shortened where the light is strong (`lit: true`).

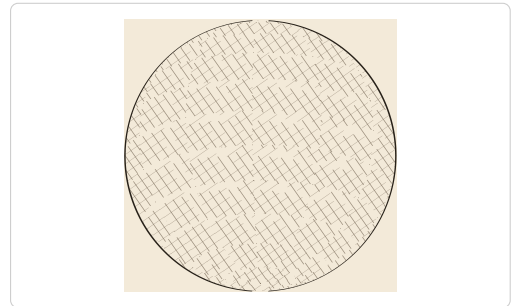
PARAMETERS

name	type	default	description
<code>spacing</code>	<code>number</code>	<code>0.24</code>	Distance between hatch lines, in projected units.
<code>angle</code>	<code>number</code>	<code>35</code>	Direction, in degrees.
<code>cross</code>	<code>bool</code>	<code>false</code>	Second family at right angles.

name	type	default	description
<code>broken</code>	<code>bool</code>	<code>true</code>	Break lines into short segments (<code>segment</code> , <code>gap</code>).
<code>lit</code>	<code>bool</code>	<code>true</code>	Hatch only the shaded part.
<code>segment</code> , <code>gap</code>	<code>number</code>	<code>0.22</code> , <code>0.08</code>	Length of segments and gaps (broken lines).
<code>count</code> , <code>length</code> , <code>crosshatch</code> , <code>seed</code> ...		<code>none</code> , <code>none</code> , <code>none</code> , <code>11</code>	Kept for compatibility, ignored since 0.4.
<code>pen</code>	<code>pen</code>	<code>pale sepia</code>	Pen of the hatch lines.

EXAMPLE

```
let s = sphere(1)
picture(width: 3.6cm, projection: orthographic(4,
2, 1.6), style: "vintage",
  draw(surface(s), hatch: vintage-hatch(spacing:
0.07, cross: true, lit: false)),
  draw(silhouette(s)))
```



vintage-stipple

```
vintage-stipple(count: 42, seed: 0, min-size: 0.012, max-size: 0.028, gamma: 1.8,
distribution: "random", lit: true, light: auto, max-patch-count: 96, pen: ...)
```

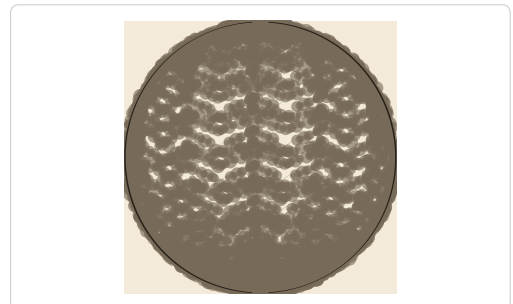
Stippling (dots) instead of hatching: dots are denser and bigger in the shadows.

PARAMETERS

name	type	default	description
<code>count</code>	<code>integer</code>	<code>42</code>	Dots per patch at full shadow.
<code>seed</code>	<code>integer</code>	<code>0</code>	Random seed.
<code>min-size</code> , <code>max-size</code>	<code>number</code>	<code>0.012</code> , <code>0.028</code>	Dot sizes (projected units).
<code>gamma</code>	<code>number</code>	<code>1.8</code>	Tone curve.
<code>distribution</code>	<code>string</code>	<code>"random"</code>	Placement of the dots.
<code>lit</code> , <code>light</code>	<code>...</code>	<code>true</code> , <code>auto</code>	As in <code>vintage-hatch</code> .
<code>max-patch-count</code>	<code>integer</code>	<code>96</code>	Patches treated at most (speed limit).

EXAMPLE

```
let s = sphere(1)
picture(width: 3.6cm, projection: orthographic(4,
2, 1.6), style: "vintage",
  draw(surface(s), stipple: vintage-stipple(count:
240, min-size: 0.02, max-size: 0.05, lit: false)),
  draw(silhouette(s)))
```



vintage-shaded / vintage-material

```
vintage-shaded(shape, hatch: auto, stipple: none, pen: auto, outline: auto) vintage-
material(color: vintage-wash, opacity: 0.82)
```

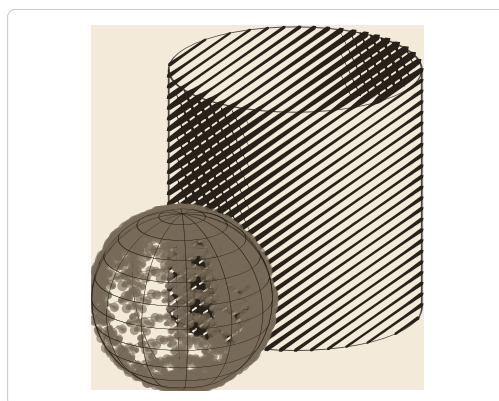
`vintage-shaded` is the vintage counterpart of `engraved`: a hatched surface and its outline in one call. `vintage-material` is the translucent sepia wash used as surface colour. Constants: `vintage-paper`, `vintage-ink`, `vintage-pale-ink`, `vintage-wash`.

PARAMETERS

name	type	default	description
<code>shape</code>	<code>shape</code>	<code>required</code>	The solid.
<code>hatch, stipple</code>	<code>spec</code>	<code>auto, none</code>	<code>auto</code> engraves the surface by tone; otherwise a <code>vintage-hatch()</code> or <code>vintage-stipple()</code> spec.
<code>pen, outline</code>	<code>pen</code>	<code>auto</code>	Hatch pen and outline pen.
<code>color, opacity</code>	<code>colour, number</code>	<code>vintage-wash, 0.82</code>	Parameters of <code>vintage-material</code> .

EXAMPLE

```
picture(width: 4.4cm, projection: orthographic(4,
2, 1.6), style: "vintage",
  vintage-shaded(cylinder(0.7, 1.4)),
  vintage-shaded(sphere(0.5, c: (1.4, 0, 0.5))),
  stipple: vintage-stipple(), hatch: none))
```



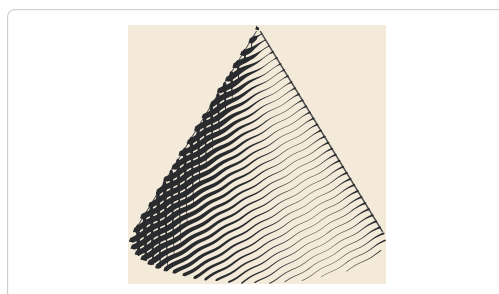
`vintage-picture` / `pencil-picture`

```
vintage-picture(..commands, ...picture options)    pencil-picture(..commands, ...picture
options)
```

`picture` with `style: "vintage"` or `style: "pencil"` already set (warm paper background). Same options as `picture`.

EXAMPLE

```
let s = cone(0.9, 1.6)
pencil-picture(width: 3.4cm, projection:
orthographic(4, 2, 1.6),
  draw(surface(s)), draw(silhouette(s)))
```



`pencil-pen`

```
pencil-pen(paint: black, thickness: 0.65pt, roughness: 0.10, passes: 1, seed: 0,
opacity: 0.82, nib-major: 1.15, nib-angle: 24.0, nib-ratio: 0.42, pressure: none,
samples: none, cap: "round", join: "round")
```

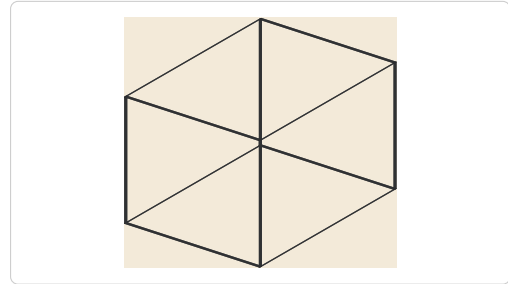
A graphite pen: an elliptical nib swept along the path with a little jitter (`roughness`) and several passes. Use it as any pen: `draw(path, pencil-pen(thickness: 1pt, passes: 2))`.

PARAMETERS

name	type	default	description
paint	colour	black	Graphite colour.
thickness	length	0.65pt	Line width.
roughness	number	0.10	Amplitude of the hand tremble.
passes	integer	1	Number of overlaid strokes.
seed	integer	0	Random seed.
opacity	number	0.82	Opacity of each pass.
nib-major, nib-angle, nib-ratio	number	1.15, 24, 0.42	Shape of the elliptical nib.
pressure, samples	...	none	Pressure profile along the path, samples.
cap, join	string	"round"	Line cap and join.

EXAMPLE

```
picture(width: 3.6cm, projection: orthographic(4,
3, 2.4), style: "pencil",
  draw(box3(0, (1.6, 1.2, 1)), pencil-
pen(thickness: 1.1pt, passes: 2, roughness: 0.2)))
```



Other tools

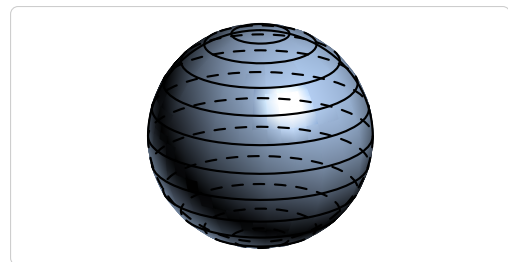
solid-plot

```
solid-plot(shape, size: 6cm, projection: currentprojection, light: currentlight, style:
"none", surfacepen: none, meshpen: none, hatch: none, stipple: none, outline: none, m:
auto, n: 12, shading: "smooth", refine: true, backend: "svg") (alias plot3d)
```

One-call figure for a solid with explicit pens: kept for compatibility; `fig` is the simpler way.

EXAMPLE

```
solid-plot(sphere(1), size: 3cm, projection:
orthographic(4, 2, 1.6),
  surfacepen: rgb("#a9c4e8"), outline:
linewidth(0.8pt))
```



obj

```
obj(text, colors: (:), transform: none)
```

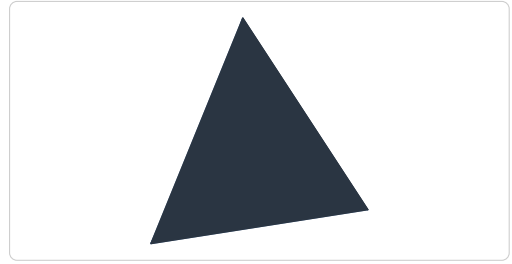
Reads the text of a Wavefront `.obj` file (`v` vertices, `f` faces, `g` / `usemtl` groups) and returns a surface. Load the file with `read("model.obj")`. `colors` maps group names to colours, `transform` is a 4×4 matrix applied to the vertices.

PARAMETERS

name	type	default	description
text	string	required	Content of the <code>.obj</code> file.
colors	dictionary	(:)	Group name $\rightarrow$ colour.
transform	transform	none	Matrix applied to the vertices.

EXAMPLE

```
let tetra = "v 0 0 0\nv 1.4 0 0\nv 0.7 1.2 0\nv 0.7  
0.4 1.3\nf 1 3 2\nf 1 2 4\nf 2 3 4\nf 3 1 4"  
fig(obj(tetra), size: 3cm, view: (15, 35))
```



Reference — knots, braces and plates

Knots and links

A strand is a function `t => (x, y, z)` on `[0, 1)` describing a **closed** curve (or an array of points). `knot3` projects the strands as tubes with true over / under crossings. In `fig` you simply give the curves; use `knot3` directly inside `picture` for several knots in one scene or to choose the colours.

knot3

```
knot3(..strands, samples: 120, width: 7pt, colors: auto, style: "gap", gap: auto,
      outline: auto, ink: auto, flips: ())
```

Draws one or several closed curves as tubes. At each crossing the strand that is **lower** (further from the camera) is interrupted (`style: "gap"`) or simply drawn first and covered by the upper one (`style: "weave"`, continuous tubes).

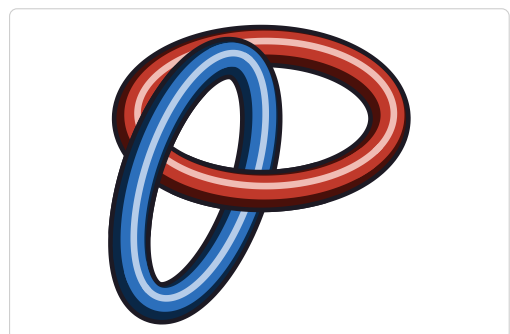
PARAMETERS

name	type	default	description
<code>..strands</code>	function or array	required	Closed curves <code>t => point</code> , or arrays of such functions (as returned by <code>hopf-link()</code> , <code>borromean()</code>).
<code>samples</code>	integer	120	Points per strand.
<code>width</code>	length	7pt	Tube diameter on the page.
<code>colors</code>	array	auto	One colour per strand (cycles).
<code>style</code>	string	"gap"	"gap" or "weave".
<code>gap</code>	length	auto	Clearance left at an interrupted crossing.
<code>outline</code>	length	auto	Width of the dark outline (<code>auto</code> = 14 % of <code>width</code>).
<code>ink</code>	colour	auto	Colour of the outline.
<code>flips</code>	array	()	Indices of crossings whose over / under order is reversed by hand (to fix a view-dependent ambiguity).

NOTE Gap-style knots with many near crossings (Borromean rings) can lose pieces: prefer `"weave"` or a thinner tube.

EXAMPLE

```
picture(width: 4cm, projection: orthographic(5, 3, 4),
  knot3(..hopf-link(R: 1.2), width: 9pt, colors:
    (rgb("#c0392b"), rgb("#2c6fbb")), style: "weave"))
```



trefoil / figure-eight / torus-knot / hopf-link / borromean

```
trefoil(scale: 1.0)      figure-eight(scale: 1.0)      torus-knot(p, q, R: 2.0, r: 0.9)
hopf-link(R: 1.0)      borromean(a: 2.0, b: 1.2)
```

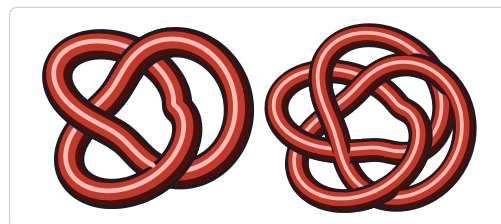
Ready-made curves. `trefoil` and `figure-eight` return one strand, `torus-knot(p, q)` the (p, q) torus knot on a torus of centre radius `R` and tube radius `r` (`p, q` coprime), `hopf-link` and `borromean` return arrays of two and three strands.

PARAMETERS

name	type	default	description
<code>scale</code>	number	<code>1.0</code>	Overall size (the trefoil is about 4 units wide).
<code>p, q</code>	integer	required	Windings around the axis / around the tube.
<code>R, r</code>	number	<code>2.0, 0.9</code>	Radii of the torus.
<code>a, b</code>	number	<code>2.0, 1.2</code>	Semi-axes of the three ellipses.

EXAMPLE

```
grid(columns: 2, gutter: 4pt,
  fig(torus-knot(2, 3), size: 2.8cm, tube: 8pt,
  view: (20, 60)),
  fig(torus-knot(3, 4, R: 2, r: 0.9), size: 2.8cm,
  tube: 6pt, view: (20, 60)))
```



brace3

```
brace3(a, b, label, flip: auto, distance: 6pt, outside: true, offset: (0, 0, 0), kind:
"brace", amplitude: 9pt, light: 0.35pt, heavy: 1.7pt, pen: auto, gap: 5pt)
```

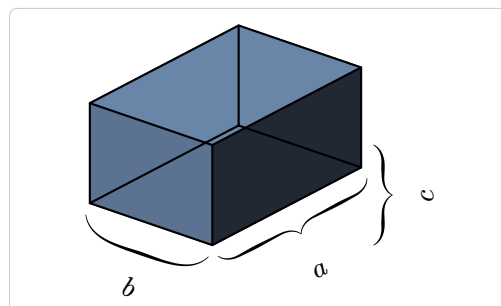
A calligraphic brace between two 3D points, drawn by nibart's `delimiter`: the pen swells where the curve bends. The label (text or formula) is centred beyond the tip. By default the brace is pushed to the **outer** side of the figure, past its bounding box, so that it never crosses the drawing; `outside: false` keeps it next to the segment.

PARAMETERS

name	type	default	description
<code>a, b</code>	point	required	Ends of the segment to measure.
<code>label</code>	content	<code>none</code>	Label (third positional argument, or <code>label:</code>).
<code>flip</code>	auto or bool	<code>auto</code>	Side of the brace: <code>auto</code> = outer side; <code>true</code> / <code>false</code> force the other / the first one.
<code>distance</code>	length	<code>6pt</code>	Gap on paper between the segment (or the figure) and the brace.
<code>outside</code>	bool	<code>true</code>	Push beyond the bounding box of the figure.
<code>offset</code>	point	<code>(0, 0, 0)</code>	3D displacement of the whole brace.
<code>kind</code>	string	<code>"brace"</code>	<code>"brace"</code> , <code>"paren"</code> or <code>"paren-straight"</code> .
<code>amplitude</code>	length	<code>9pt</code>	Height of the tip.
<code>light, heavy</code>	length	<code>0.35pt, 1.7pt</code>	Hairline and full widths of the nib.
<code>pen</code>	colour	<code>auto</code>	Ink colour (<code>auto</code> = the ink of the figure).
<code>gap</code>	length	<code>5pt</code>	Distance of the label from the tip.

EXAMPLE

```
fig(brick(3, 2, 1.4), size: 4.6cm,  
    brace3((-1.5, 1, -0.7), (1.5, 1, -0.7), $a$),  
    brace3((1.5, 1, -0.7), (1.5, -1, -0.7), $b$),  
    kind: "paren"),  
    brace3((1.5, 1, -0.7), (1.5, 1, 0.7), $c$, flip:  
    true))
```



Plates

plate

```
plate(body, title: none, subtitle: none, number: none, caption: none, ink: ..., paper: ...,  
    pad: 16pt, band: 2.2pt, corner: 9pt, border: true, width: auto, font: auto, ornament:  
    true)
```

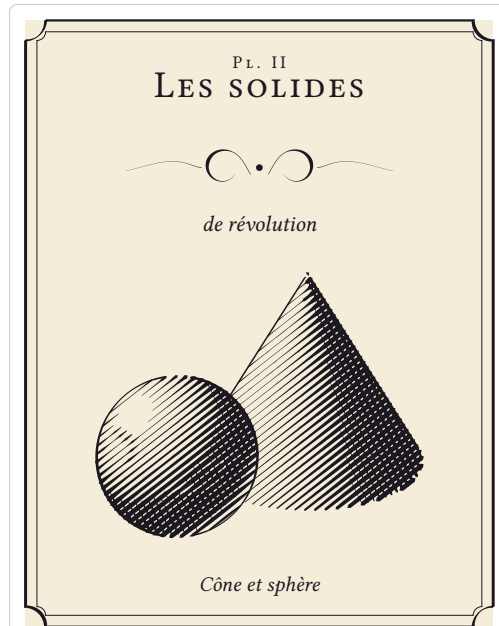
An engraved plate around `body`: cream paper, a double line frame with notched corners (drawn by `nibart`), a title with plate number (`Pl. I`), a calligraphic flourish and a caption. Use it for figure sheets in a book or a course.

PARAMETERS

name	type	default	description
<code>body</code>	content	required	What is framed.
<code>title, subtitle</code>	content	none	Heading(s) above the figure (small capitals).
<code>number</code>	content	none	Plate number (<code>Pl. I ...</code>).
<code>caption</code>	content	none	Italic caption below.
<code>ink, paper</code>	colour	engraving colours	Ink and paper colours.
<code>pad</code>	length	16pt	Space between the frame and the content.
<code>band</code>	length	2.2pt	Width of the frame band (keep it slim).
<code>corner</code>	length	9pt	Size of the corner notches.
<code>border</code>	bool	true	Draw the frame.
<code>width</code>	length, ratio or auto	auto	Total width.
<code>font</code>	string or array	auto	Font of the titles.
<code>ornament</code>	bool	true	Draw the flourish under the title.

EXAMPLE

```
plate(title: "Les solides", subtitle: "de  
révolution", number: "II", width: 7.4cm, band:  
1.6pt,  
caption: [Cône et sphère],  
fig(cone(1, 1.8), sphere(0.7, c: (1.8, 0, 0.7)),  
size: 5.2cm, style: "engraving"))
```



Reference — school geometry

Annotations and ready-made solids for geometry lessons. The annotation functions return commands: give them to `fig` or `picture` together with the solid. The projected label of a dimension follows the line; hidden edges are dashed (dashed is a pen).

Solids for lessons

`cuboid` / `regular-pyramid` / `regular-prism`

```
cuboid(origin: 0, length: 3, width: 2, height: 1.5)    regular-pyramid(center: 0,
radius: 1, height: 2, n: 4, angle: 45)    regular-prism(center: 0, radius: 1, height:
2, n: 6, angle: 0)
```

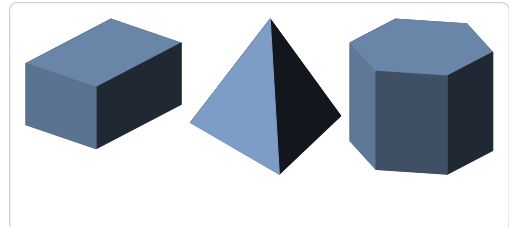
A cuboid (also `pave-droit`), a regular pyramid and a regular prism with `n`-gon base (true polygons, not circle approximations: `n = 3, 4, 6` give triangles, squares and hexagons). `radius` is the circumradius of the base, `angle` the rotation of the base in degrees.

PARAMETERS

name	type	default	description
<code>origin / center</code>	<code>point</code>	<code>0</code>	Lower corner of the cuboid / centre of the base.
<code>length, width, height</code>	<code>number</code>	<code>3, 2, 1.5</code>	Dimensions of the cuboid (x, y, z).
<code>radius</code>	<code>number</code>	<code>1</code>	Circumradius of the base.
<code>height</code>	<code>number</code>	<code>2</code>	Height.
<code>n</code>	<code>integer</code>	<code>4 / 6</code>	Sides of the base.
<code>angle</code>	<code>number</code>	<code>45 / 0</code>	Rotation of the base (degrees).

EXAMPLE

```
grid(columns: 3, gutter: 4pt,
  fig(cuboid(), size: 2.6cm),
  fig(regular-pyramid(n: 3, radius: 1.2, height:
1.8), size: 2.6cm),
  fig(regular-prism(n: 6, height: 1.5), size:
2.6cm))
```



`regular-tetrahedron` / `regular-octahedron` / `frustum3` / `right-prism3`

```
regular-tetrahedron(center: 0, radius: 1)    regular-octahedron(center: 0, radius: 1)
frustum3(center: 0, bottom-radius: 1, top-radius: 0.6, height: 1.5, n: 24)    right-
prism3(origin: 0, base: 2, width: 1.2, height: 2)
```

More solids: two Platonic solids (circumradius `radius`), a frustum of a cone, and a right prism whose base is a right triangle with legs `base` and `width`.

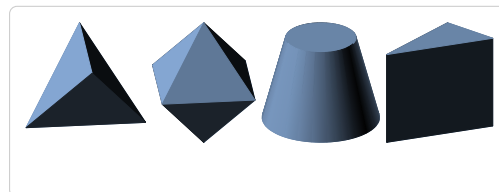
PARAMETERS

name	type	default	description
<code>center</code>	<code>point</code>	<code>0</code>	Centre of the (bottom) base.
<code>radius</code>	<code>number</code>	<code>1</code>	Circumradius.
<code>bottom-radius, top-radius</code>	<code>number</code>	<code>1, 0.6</code>	Radii of the frustum.
<code>height</code>	<code>number</code>	<code>1.5 / 2</code>	Height.
<code>n</code>	<code>integer</code>	<code>24</code>	Slices of the frustum.

name	type	default	description
<code>origin</code> , <code>base</code> , <code>width</code>	<code>point</code> , <code>number</code>	<code>0</code> , <code>2</code> , <code>1.2</code>	Corner and legs of the right prism.

EXAMPLE

```
grid(columns: 4, gutter: 3pt,
  fig(regular-tetrahedron(), size: 2.1cm),
  fig(regular-octahedron(), size: 2.1cm),
  fig(frustum3(), size: 2.1cm),
  fig(right-prism3(), size: 2.1cm))
```



Annotations

point3 / vertex3

```
point3(name, p, align: E, pen: auto, size: auto, dx: 0, dy: 0)
```

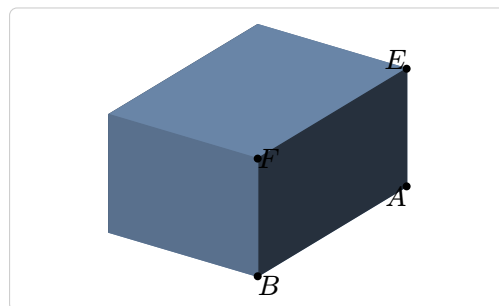
Marks a point with a dot and names it. `align` is a **projected** direction (N, S, E, W, NE, NW, SE, SW); `dx`, `dy` nudge the label (in projected units or in lengths such as 2pt).

PARAMETERS

name	type	default	description
<code>name</code>	<code>content</code>	required	Name of the point (<code>\$A\$</code>).
<code>p</code>	<code>point</code>	required	Position.
<code>align</code>	<code>direction</code>	<code>E</code>	Where the label goes.
<code>pen</code> , <code>size</code>	<code>pen</code> , <code>length</code>	<code>auto</code>	Colour and dot diameter.
<code>dx</code> , <code>dy</code>	<code>number</code> or <code>length</code>	<code>0</code>	Fine adjustment.

EXAMPLE

```
fig(cuboid(length: 2, width: 1.4, height: 1), size:
4cm, view: (35, 25),
  point3($A$, (0, 1.4, 0), align: SW), point3($B$,
(2, 1.4, 0), align: SE),
  point3($F$, (2, 1.4, 1), align: E), point3($E$,
(0, 1.4, 1), align: NW))
```



dimension3

```
dimension3(text, a, b, offset: 0, align: N, pen: auto, arrow: Arrows, rotate: auto, dx:
0, dy: 0, extensions: true, extension-pen: auto) (alias length3-label, segment-
label3)
```

A dimension line between `a` and `b`, translated by the 3D vector `offset` (so that it stands away from the edge), with extension lines and arrows at both ends and a label that follows the line.

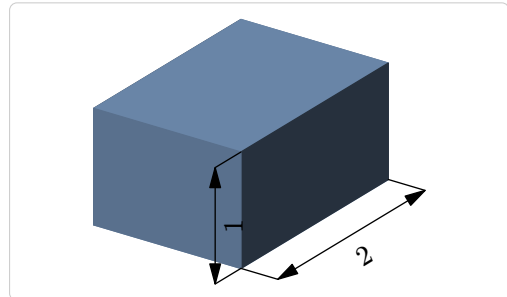
PARAMETERS

name	type	default	description
<code>text</code>	<code>content</code>	required	The measure: <code>\$5\$ cm</code> .
<code>a</code> , <code>b</code>	<code>point</code>	required	Ends of the measured segment.
<code>offset</code>	<code>point</code>	<code>0</code>	3D displacement of the dimension line.
<code>align</code>	<code>direction</code>	<code>N</code>	Side of the label.

name	type	default	description
<code>pen</code>	<code>pen</code>	<code>auto</code>	Dimension line and arrowheads.
<code>arrow</code>	<code>arrow</code>	<code>Arrows</code>	Arrowheads (<code>Arrows</code> , <code>Arrow3</code> , <code>none</code> ...).
<code>rotate</code>	<code>auto, false or angle</code>	<code>auto</code>	Label follows the line / stays horizontal / forced angle.
<code>dx, dy</code>	<code>number</code>	<code>0</code>	Move the label only.
<code>extensions</code>	<code>bool</code>	<code>true</code>	Draw the extension lines.
<code>extension-pen</code>	<code>pen</code>	<code>auto</code>	Pen of the extension lines (e.g. <code>dotted</code>).

EXAMPLE

```
fig(cuboid(length: 2, width: 1.4, height: 1), size:
4.4cm, view: (35, 25),
  dimension3($2$, (0, 1.4, 0), (2, 1.4, 0), offset:
(0, 0.35, 0), align: S),
  dimension3($1$, (2, 1.4, 0), (2, 1.4, 1), offset:
(0.35, 0, 0), align: E))
```



callout3

```
callout3(text, target, label-at, align: E, pen: auto, arrow: BeginArrow, dx: 0, dy: 0)
```

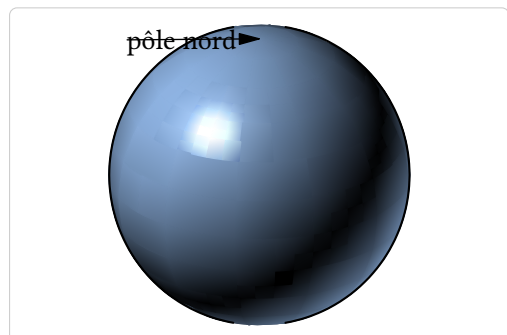
A text with a leader line whose arrow points at `target`; the text sits at `label-at`.

PARAMETERS

name	type	default	description
<code>text</code>	<code>content</code>	required	The remark.
<code>target</code>	<code>point</code>	required	Where the arrow points.
<code>label-at</code>	<code>point</code>	required	Where the text sits.
<code>align, pen, arrow, dx, dy</code>	...	<code>E, auto, BeginArrow, 0, 0</code>	As in <code>point3</code> .

EXAMPLE

```
fig(sphere(1), size: 4cm, callout3([pôle nord], (0,
0, 1), (1.4, 0, 1.5), align: E))
```



edge-code3 / equal-edges3

```
edge-code3(a, b, count: 1, size: 0.14, spacing: 0.16, at: 0.5, direction: auto, pen:
auto)    equal-edges3(edges, count: 1, size: 0.14, spacing: 0.16, at: 0.5, direction:
auto, pen: auto)
```

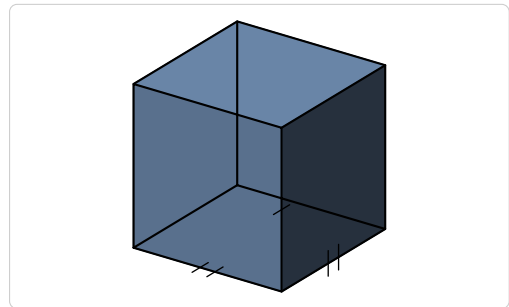
Tick marks (1, 2 or 3) on an edge to show equal lengths. `equal-edges3` takes a list of edges `((a, b), (c, d), ...)` and marks all of them the same way. The ticks are built in 3D and projected with the figure.

PARAMETERS

name	type	default	description
a, b / edges	points / array	required	Ends of the edge / list of edges.
count	integer	1	Number of ticks.
size	number	0.14	Length of a tick (3D units).
spacing	number	0.16	Distance between ticks.
at	number	0.5	Position along the edge (0 – 1).
direction	point	auto	Direction of the ticks (default: perpendicular, in the face).
pen	pen	auto	Pen.

EXAMPLE

```
fig(cube(1.6), size: 3.6cm, view: (35, 25),
    equal-edges3(((0.8, 0.8, -0.8), (0.8, 0.8,
-0.8)), ((0.8, 0.8, -0.8), (0.8, -0.8, -0.8))),
count: 2, size: 0.25),
    edge-code3((0.8, 0.8, -0.8), (0.8, 0.8, 0.8),
count: 1, size: 0.25))
```



right-angle3 / angle-mark3 / angle-code3

```
right-angle3(a, b, c, size: 0.15, pen: auto)    angle-mark3(a, b, c, radius: 0.24, pen:
auto, label: none, align: N)    angle-code3(a, b, c, right: false, size: 0.15, radius:
0.24, pen: auto, label: none, align: N)
```

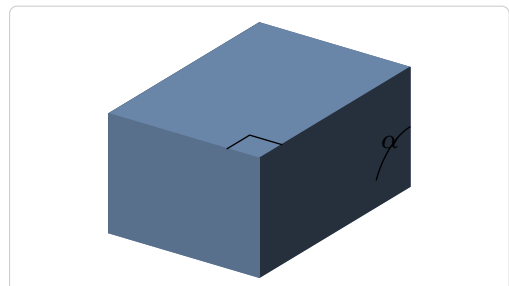
Marks the angle at vertex **b** between rays **ba** and **bc**: a small square (right angle), an arc with an optional label, or either of them (**angle-code3** with **right: true / false**).

PARAMETERS

name	type	default	description
a, b, c	point	required	Points; b is the vertex.
size	number	0.15	Side of the right-angle square.
radius	number	0.24	Radius of the arc.
right	bool	false	angle-code3 : square instead of arc.
label, align	content, direction	none, N	Text near the arc.
pen	pen	auto	Pen.

EXAMPLE

```
fig(cuboid(length: 2, width: 1.4, height: 1), size:
4cm, view: (35, 25),
    right-angle3((0, 1.4, 1), (2, 1.4, 1), (2, 0, 1),
size: 0.3),
    angle-mark3((0, 1.4, 1), (0, 1.4, 0), (2, 1.4,
1), label: '$\alpha$', radius: 0.5))
```



construction3 / height3 / diagonal3 / projection3

```
construction3(a, b, pen: auto)    height3(top, foot, pen: dashed, mark: false, mark-
size: 0.12)    diagonal3(a, b, pen: dashed)    projection3(p, foot, pen: auto)
```

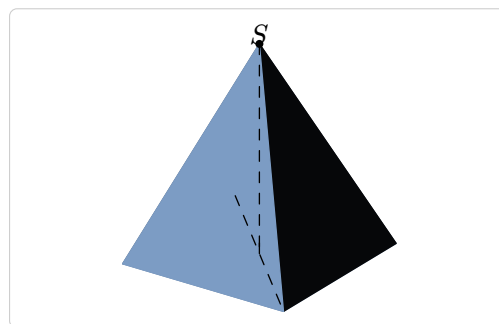
Dashed construction lines: any segment, the height of a solid with an optional right-angle mark at the foot, a diagonal, the perpendicular projection of a point.

PARAMETERS

name	type	default	description
a, b / top, foot / p	point	required	Ends.
pen	pen	dashed	Pen (dashed, dotted, a colour...).
mark	bool	false	height3: small right-angle mark at the foot.
mark-size	number	0.12	Size of that mark.

EXAMPLE

```
fig(regular-pyramid(n: 4, radius: 1.2, height: 2),
size: 3.8cm, view: (35, 25),
height3((0, 0, 2), 0, mark: true),
diagonal3((-0.85, -0.85, 0), (0.85, 0.85, 0)),
point3($S$, (0, 0, 2), align: N))
```



cuboid-vertices / cuboid-edges

```
cuboid-vertices(origin: 0, length: 3, width: 2, height: 1.5)    cuboid-edges(v,
visible: black, hidden: dashed)
```

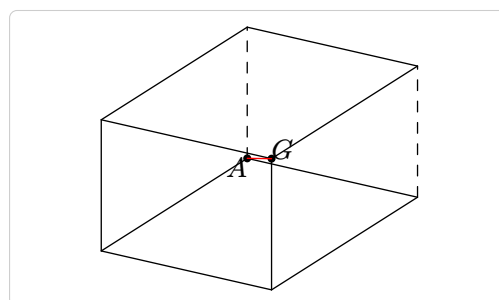
`cuboid-vertices` returns a dictionary of the eight corners A ... H (ABCD the bottom, EFGH the top, E above A); `cuboid-edges` draws the twelve edges with the hidden ones in another pen. Handy to name vertices and to compute with them.

PARAMETERS

name	type	default	description
origin, length, width, height	...	0, 3, 2, 1.5	As in <code>cuboid</code> .
v	dictionary	required	Result of <code>cuboid-vertices</code> .
visible, hidden	pen	black, dashed	Pens of the visible / hidden edges.

EXAMPLE

```
let v = cuboid-vertices(length: 2, width: 1.4,
height: 1)
picture(width: 4.2cm, projection: orthographic(5,
3, 2.4),
cuboid-edges(v),
point3($A$, v.A, align: SW), point3($G$, v.G,
align: NE),
diagonal3(v.A, v.G, pen: red))
```



Appendices

Performance and memory

Figures are computed in pure Typst, so their cost is that of a Typst script. Rough guide on a laptop: a shaded `fig` takes 0.2 – 0.6 s; an `engraving` or `banknote` figure 1 – 3 s; knots about 0.3 s. Typst keeps every computed figure in memory until the end of the compilation (about 40 MB for a shaded sphere): a document with more than 60 figures on a machine with 2 GB of memory should be compiled in pieces, or use the tips below.

- Lower the smoothness: `sphere(1, n: 8)`, `surface-grid(..., nu: 16, nv: 12)`.
- Use shading: "flat" or `refine: false` in `fig` / `picture` for drafts.
- Engraving: increase `spacing` (e.g. 3pt) and `step` (2.4pt); `cross: false` halves the work.
- Reuse a figure: compute it once with `let f = fig(...)` and place `f` several times (Typst caches the result).
- The manual of this package is built chapter by chapter (`docs/build-manual.py`) for that reason.

Troubleshooting

Symptom	Cause and remedy
A solid is black	A surface without colour is black: with <code>fig</code> use <code>paint(shape, colour)</code> ; with <code>draw</code> give a colour or material.
A solid hides another one wrongly	Surfaces are painted in the order given. Put the farthest first, or use <code>picture(zsort: "global")</code> when solids interpenetrate.
<code>cannot compare float.nan</code>	A point contains <code>nan</code> or an axis <code>min</code> / <code>max</code> was given as a point: <code>min</code> / <code>max</code> of <code>xaxis3</code> are numbers.
<code>number must be at least zero</code>	<code>-1 * (1, 1, 1)</code> repeats an array in Typst; write <code>(-1, -1, -1)</code> or <code>vmul(-1, v)</code> .
A knot loses pieces or has wrong crossings	Use <code>knot-style: "weave"</code> , a thinner <code>tube</code> , a view that separates the strands, or <code>flips</code> in <code>knot3</code> .
A brace crosses the figure	Force the other side with <code>flip: true</code> / <code>false</code> , or move it with <code>offset</code> .
<code>package not found: nibart</code>	Edition mismatch: the Universe edition needs <code>@preview/nibart:0.3.0</code> , the local edition <code>@local/nibart:0.3.0</code> (install <code>nibart-local</code> first).
Compilation is killed (memory)	See "Performance and memory" above.

Coming from Asymptote

The advanced layer keeps Asymptote's names and conventions: `currentprojection`, `orthographic`, `perspective`, `draw`, `dot`, `label`, `path3` operations (`subpath`, `arclength`, `arctime...`), `revolution`, `surface`, `light`, `material`, `pen`, `dashed`, `Arrow3`. Differences: functions take named arguments in Typst syntax (`draw(p, linewidth(1pt))`), pens combine with `pen(a, b)` rather than `+`, a picture is a Typst box and several pictures are laid out with `grid`; angles of the vector helpers are in degrees. The French guide (`docs/guide-avance-fr.pdf`) contains the full correspondence table.

What's new in 0.3

- **fig**: the one-call simple API (view names, styles, automatic light, one link for all curves), with `paint`, `cube`, `brick` and `view-projection`.
- **Styles** `banknote` and `etching` in `fig`; every solid constructor accepts `c:` for its centre.

- **brace3** rewritten: outer side chosen automatically (`flip: auto`), `outside`, `distance`, third positional argument for the label.
- **engraved** takes a plain colour; exact torus silhouette (0.2.0 fix kept).
- **Manual** rewritten (this document, English + French) with every function, its parameters and an example; the former French manual becomes the **advanced guide** (`docs/guide-avance-fr.pdf`).
- Author and licence metadata; Universe and local editions.

Licences and credits

solids3d is released under the LGPL-3.0-or-later (see `LICENSE`). It uses **nibart** (MIT, FERGOUS Abdelhak) for pens, outlines, knots, braces and plates. The package contains no binary and no third-party code. The API follows the vocabulary of Asymptote's `three` and `solids` modules; no Asymptote code is included.